

AMBA Design Kit

Technical Reference Manual



AMBA Design Kit

Technical Reference Manual

Copyright © 2002 ARM Limited. All rights reserved.

Release information

Change history

Date	Issue	Change
January 2002	A-01	First draft. Based on DDI 0219B.

Proprietary notice

Words and logos marked with ® or ™ are registered trademarks or trademarks owned by ARM Limited, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Confidential. This information can only be distributed to ARM employees, and employees of companies that have signed a Non-Disclosure Agreement (NDA).

Product status

The information in this document is Preliminary (information on a product under development).

ARM web address

<http://www.arm.com>

Contents

AMBA Design Kit Technical Reference Manual

	Preface	
	About this document	vi
	Further reading.....	ix
	Feedback	x
Chapter 1	Introduction	
	1.1 About the ADK	1-2
	1.2 Address map	1-3
	1.3 AMBA signals.....	1-4
Chapter 2	Functional Overview	
	2.1 About the ADK	2-2
	2.2 ADK components	2-3
	2.3 Example systems	2-6
Chapter 3	AHB Components	
	3.1 Reset controller (<i>ResetCntl</i>).....	3-2
	3.2 Arbiter (<i>Arbiter3</i>)	3-3
	3.3 Default slave (<i>DefaultSlave</i>).....	3-4
	3.4 Master to slave multiplexor (<i>MuxM2S</i>).....	3-5
	3.5 Slave to master multiplexor (<i>MuxS2M</i>)	3-6
	3.6 Retry slave	3-7
	3.7 Static memory interface (<i>SMI</i>).....	3-8

3.8	Multi-layer bus switch (<i>BusMatrix</i>)	3-17
3.9	Decoder (<i>Decoder</i>)	3-19
3.10	APB bridge (<i>APBif</i>)	3-22
3.11	Example bus master	3-24
3.12	AHB to AHB bridge (<i>Ahb2Ahb</i>)	3-25
3.13	AHB-Lite to AHB wrapper	3-34
3.14	Interrupt controller (<i>Interrupt</i>)	3-36
Chapter 4	APB Components	
4.1	Remap and pause controller (<i>RemapPause</i>)	4-2
4.2	Example APB slave	4-10
4.3	Peripheral to bridge multiplexor (<i>MuxP2B</i>)	4-13
4.4	Watchdog unit (<i>Watchdog</i>)	4-14
4.5	Dual input timer (<i>Timers</i>)	4-24
Chapter 5	Behavioral Models	
5.1	External RAM (<i>ExtRAM</i>)	5-2
5.2	Internal memory (<i>IntMem</i>)	5-3
5.3	External ROM (<i>ExtROM</i>)	5-5
5.4	Tube (<i>Tube</i>)	5-6
5.5	File reader bus master (<i>FileReader</i>)	5-7
5.6	TIC driver (<i>TIC</i>)	5-10
Chapter 6	PrimeCell GPIO	
6.1	Operation	6-2
6.2	Integration within ADK	6-3
Appendix A	AHB-Lite Overview	
A.1	Introduction	A-2
A.2	AHB-Lite master	A-4
A.3	AHB-Lite slaves	A-5

Preface

This preface introduces the ARM *AMBA Design Kit* (ADK) and its reference documentation. It contains the following sections:

- *About this document* on page vi
- *Further reading* on page ix
- *Feedback* on page x.

About this document

This document is the technical reference manual for the ADK.

Intended audience

This document has been written for *System-on-Chip* (SoC) designers and system architects, and provides a description of components within the ADK architecture.

Using this manual

This document is organized into the following chapters:

Chapter 1 *Introduction*

Read this chapter for an introduction to the ADK.

Chapter 2 *Functional Overview*

Read this chapter for an overview of the top level structure of the ADK, and examples of how the ADK can be used.

Chapter 3 *AHB Components*

Read this chapter for a description of the AHB components used in the ADK.

Chapter 4 *APB Components*

Read this chapter for a description of the APB components used in the ADK.

Chapter 5 *Behavioral Models*

Read this chapter for a description of the behavioral models in the ADK.

Chapter 6 *PrimeCell GPIO*

Read this chapter for a description of how the PrimeCell *General Purpose Input/Output* (GPIO) is integrated within the ADK.

Appendix A *AHB-Lite Overview*

Read this appendix for an overview of the AHB-Lite, which is a subset of the full AHB specification.

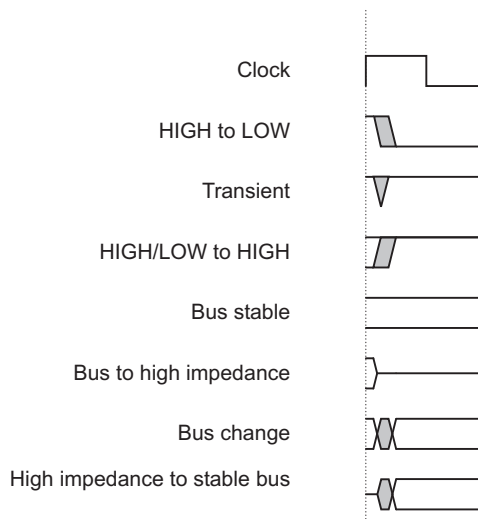
Typographical conventions

The following typographical conventions are used in this document:

bold	Highlights ARM processor signal names, and interface elements such as menu names. Also used for terms in descriptive lists, where appropriate.
<i>italic</i>	Highlights special terminology, cross-references, and citations.
<code>monospace</code>	Denotes text that can be entered at the keyboard, such as commands, file names and program names, and source code.
<u>monospace</u>	Denotes a permitted abbreviation for a command or option. The underlined text can be entered instead of the full command or option name.
<code>monospace italic</code>	Denotes arguments to commands or functions where the argument is to be replaced by a specific value.
monospace bold	Denotes language keywords when used outside example code.

Timing diagram conventions

This manual contains one or more timing diagrams. The following key explains the components used in these diagrams. Any variations are clearly labeled when they occur. Therefore, no additional meaning must be attached unless specifically stated.



Key to timing diagram conventions

Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.

Further reading

This section lists publications by ARM Limited, and by third parties.

ARM periodically provides updates and corrections to its documentation. See <http://www.arm.com> for current errata sheets and addenda.

See also the ARM Frequently Asked Questions list at:
<http://www.arm.com/DevSupp/Sales+Support/faq.html>

ARM publications

This document contains information that is specific to the ADK. Refer to the following documents for other relevant information:

- *AHB Example AMBA System Technical Reference Manual* (ARM DDI 0170)
- *ARM PrimeCell General Purpose Input/Output (PL061) Technical Reference Manual* (ARM DDI 0190)
- *ADK Design Guide* (ARM DUI 0183).

Feedback

ARM Limited welcomes feedback both on the ADK, and on the documentation.

Feedback on the ADK

If you have any comments or suggestions about this product, contact your supplier giving:

- the product name
- a concise explanation of your comments.

Feedback on this document

If you have any comments about this document, send an email to errata@arm.com giving:

- the document title
- the document number
- the page number(s) to which your comments refer
- a concise explanation of your comments.

General suggestions for additions and improvements are also welcome.

Chapter 1

Introduction

This chapter describes the *AMBA Design Kit* (ADK). It contains the following sections:

- *About the ADK* on page 1-2
- *Address map* on page 1-3
- *AMBA signals* on page 1-4.

1.1 About the ADK

The ADK comprises the building blocks required to create an example system based on the low-power, generic design methodology of the *Advanced Microcontroller Bus Architecture* (AMBA). Three preconfigured and validated examples enable custom devices to be developed in very short design cycles. This allows the resulting subcomponents to be easily reused in future designs.

1.2 Address map

Figure 1-1 shows a block diagram of the ADK address map.

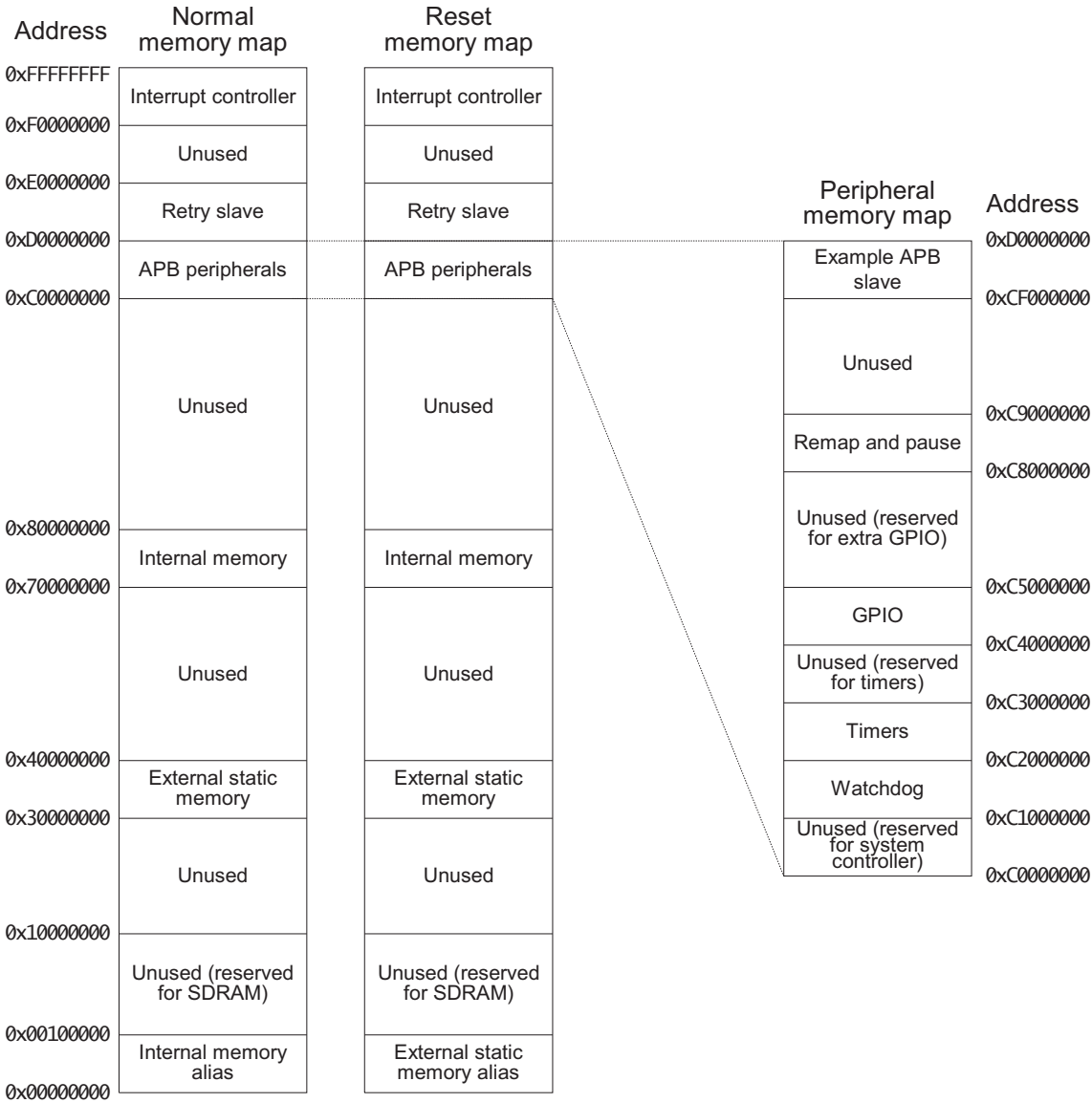


Figure 1-1 ADK address map

1.3 AMBA signals

- Two sets of signals are described:
- *AMBA AHB signals* on page 1-4
 - *AMBA APB signals* on page 1-6.

———— **Note** ————

For a description of the non-AMBA signals used in the ADK, see the applicable section in Chapter 3, 4, and 5.

1.3.1 AMBA AHB signals

Table 1-1 contains a list of AMBA AHB signals used in the ADK.

Table 1-1 AMBA AHB signal descriptions

Signal	Direction				Description
	Slave	Master	Arbiter	Decoder	
HCLK	Input	Input	Input	-	This clock times all bus transfers.
HRESETn	Input	Input	Input	Input	The bus reset signal is active LOW, and is used to reset the system and the bus.
HSEL	Input	-	-	Output	Each AHB slave has its own slave select signal and this signal indicates that the current transfer is intended for the selected slave. This signal is simply a combinatorial decode of the address bus.
HTRANS[1:0]	Input	Output	Input	-	This indicates the type of the current transfer, which can be NONSEQUENTIAL, SEQUENTIAL, IDLE, or BUSY.
HADDR[31:0]	Input	Output	-	Input	The 32-bit system address bus.
HWRITE	Input	Output	-	-	When HIGH, this signal indicates a write transfer, and when LOW, a read transfer.
HSIZE[2:0]	Input	Output	-	-	These signals indicate the size of the transfer, which is typically byte (8-bit), halfword (16-bit) or word (32-bit). The protocol allows for larger transfer sizes up to a maximum of 1024 bits.
HBURST[2:0]	Input	Output	Input	-	These signals indicate if the transfer forms part of a burst. Four, eight, and sixteen beat bursts are supported and the burst can be either incrementing or wrapping.

Table 1-1 AMBA AHB signal descriptions (continued)

Signal	Direction				Description
	Slave	Master	Arbiter	Decoder	
HPROT[3:0]	Input	Output	-	-	The protection control signals provide additional information about a bus access and are primarily intended for use by any module that wishes to implement some level of protection. The signals indicate if the transfer is an opcode fetch or data access, as well as if the transfer is a privileged mode access or User mode access. For bus masters with a memory management unit these signals also indicate whether the current access is cacheable or bufferable.
HMASTLOCK	Input	-	Output	-	Indicates that the current master is performing a locked sequence of transfers. This signal has the same timing as the HMASTER signals.
HLOCK	-	Output	Input	-	When HIGH, this signal indicates that the master requires locked access to the bus and no other master must be granted the bus until this signal is LOW.
HBUSREQ	-	Output	Input	-	A signal from bus master <i>x</i> to the bus arbiter which indicates that the bus master requires the bus. There is an HBUSREQ signal for each bus master in the system, up to a maximum of 16 bus masters.
HMASTER [3:0]	Input	-	Output	-	These signals from the arbiter indicate which bus master is currently performing a transfer and is used by the slaves that support SPLIT transfers to determine which master is attempting an access. The timing of HMASTER is aligned with the timing of the address and control signals.
HWDATA [31:0]	Input	Output	-	-	The write data bus is used to transfer data from the master to the bus slaves during write operations. A minimum data bus width of 32 bits is recommended. However, this can easily be extended to allow for higher bandwidth operation.
HREADY HREADYOUT	Input Output	Input -	Input -	- -	When HIGH, the HREADY signal indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.
HRESP[1:0]	Output	Input	Input	-	The transfer response provides additional information on the status of a transfer. Four different responses are provided, OKAY, ERROR, RETRY, and SPLIT.

Table 1-1 AMBA AHB signal descriptions (continued)

Signal	Direction				Description
	Slave	Master	Arbiter	Decoder	
HRDATA[31:0]	Output	Input	-	-	The read data bus is used to transfer data from bus slaves to the bus master during read operations. A minimum data bus width of 32 bits is recommended. However, this can easily be extended to allow for higher bandwidth operation.
HGRANT	-	Input	Output	-	This signal indicates that the bus master is currently the highest priority master. Ownership of the address/control signals changes at the end of a transfer when HREADY is HIGH, so the master gets access to the bus when both HREADY and HGRANT are HIGH.
HSPLIT[15:0]	Output	-	Input	-	The 16-bit split bus is used by a split-capable slave to indicate to the arbiter which bus masters are allowed to reattempt a split transaction. Each bit of this split bus corresponds to a single bus master.

1.3.2 AMBA APB signals

Table 1-2 contains a list of AMBA APB signals used in the ADK.

Table 1-2 ADK AMBA APB signal descriptions

Signal	Type	Direction	Description
PCLK	Peripheral clock	Input	This clock times all bus transfers. Both the LOW phase and HIGH phase of PCLK are used to control transfers.
PRESETn	Peripheral reset	Input	The bus reset signal is active LOW and is used to reset the system.
PADDR[31:0]	Peripheral address bus	Output	This is the APB address bus, which can be up to 32 bits wide and is used by individual peripherals for decoding register accesses to that peripheral. The address becomes valid after the first rising edge of the clock at the start of the transfer. If there is a following APB transfer, the address changes to the new value, otherwise it holds its current value until the start of the next APB transfer.

Table 1-2 ADK AMBA APB signal descriptions (continued)

Signal	Type	Direction	Description
PSELx	Peripheral slave select	Output	There is one of these signals for each APB peripheral present in the system. The signal indicates that the slave device is selected, and that a data transfer is required. It has the same timing as the peripheral address bus. It becomes HIGH at the same time as PADDR , but is set LOW at the end of the transfer.
PENABLE	Peripheral enable	Output	This enable signal is used to time all accesses on the peripheral bus. PENABLE goes HIGH on the second clock rising edge of the transfer, and LOW on the third (last) rising clock edge of the transfer.
PWRITE	Peripheral transfer direction	Output	This signal indicates a write to a peripheral when HIGH , and a read from a peripheral when LOW . It has the same timing as the peripheral address bus.
PRDATA[31:0]	Peripheral read data bus	Input	The peripheral read data bus is driven by the selected peripheral bus slave during read cycles (when PWRITE is LOW).
PWDATA[31:0]	Peripheral write data bus	Output	The peripheral write data bus is continuously driven by this module, changing during write cycles (when PWRITE is HIGH).

Chapter 2

Functional Overview

This chapter describes the top level structure of the *AMBA Design Kit* (ADK), and shows a number of examples of how the ADK can be used. It contains the following sections:

- *About the ADK* on page 2-2
- *ADK components* on page 2-3
- *Example systems* on page 2-6.

2.1 About the ADK

The ADK enables designers to create AMBA-based components and *System-on-Chip* (SoC) designs. The ADK toolkit provides the SoC designer with the following capabilities:

- A fully working simulation environment to enable you to become familiar with the AMBA protocol.
- A development environment for AMBA modules.
- A number of different starting preconfigured arrangements from which complex SoC designs can be built.
- VHDL and Verilog language support.

2.2 ADK components

The ADK is made up of a number of components, which are summarized in the following sections:

- *AHB components* on page 2-3
- *APB components* on page 2-5.

2.2.1 AHB components

The AHB components are described in more detail in Chapter 3 *AHB Components*.

Arbiter

- Uses a simple priority algorithm to control which master has access to the AHB bus
- Two versions are available within the ADK:
 - three masters plus dummy
 - fifteen masters plus dummy.
- Arbitration scheme defined in a separate sub-block for easy modification and reuse.

AHB to APB bridge

- Provides a 16-slot interface between the high performance pipelined AHB and the lower performance peripheral bus.

Internal memory

- 32-bit wide on-chip SRAM
- Memory can be initialized from an ASCII file
- Variable size (default 1kB).

Retry slave

- A starting point for a typical AHB slave design
- Simple 4-word register bank plus read-only registers providing arithmetic combinations of the writable registers
- Higher order address bits control the response
- Supports retry response and wait states.

Static memory interface

- AHB to access the external RAM and ROM
- Intended as a starting point
- Replaceable with PrimeCell *Static Memory Controller* (SMC) (PL092).

Test interface controller

- Now included within the static memory interface
- Converts test vectors applied on the external pins of the device into valid AHB transfers on an internal bus
- To be used to test the ARM processor core.

File reader bus master

- Allows the generation of legal AHB transactions from an external stimulus file
- Interfaces through the AHB-Lite wrapper.

Example bus master

- A simple bus master that is intended to act as the starting point for bus master development
- Demonstrates bus master activity by read-pause-write bursts
- Interfaces through the AHB-Lite wrapper.
- Configurable transfers.

AHB to AHB bridge

- Provides an interface between two separate AHB buses.

Multi-layer bus switch

- Allows parallel access paths between multiple masters and slaves
- Improves overall systems bandwidth
- Gives increased system design flexibility.

Reset controller

- Generates a system-wide reset from power-on reset and watchdog reset signals.

Interrupt controller

- Generates the two ARM interrupt signals from the multiple interrupt sources that can exist in a system
- Individual bit level control of the masking of interrupts
- Interrupt driven or polled method of operation
- Pin compatible with the PrimeCell *Vectored Interrupt Controller* (VIC) (PL190)
- Can be daisy chained with the PrimeCell VIC.

2.2.2 APB components

The APB components are described in more detail in Chapter 4 *APB Components*.

Timers

- Two 32-bit counters with an optional prescaler up to 8-bits
- Three modes of operation:
 - free running
 - periodic interrupt
 - one shot
- Default mode gives identical specification to the timers implemented in the ARM Development Board.

Remap/pause controller

- Basic *glue logic* functions that are required to implement correct boot up behavior and a basic low-power mode of operation.

Example APB slave

- A starting point for a typical APB slave design
- Simple 4-word register bank plus read-only registers providing arithmetic combinations of the writable registers.

PrimeCell *General Purpose Input/Output* (GPIO) (PL061)

- Sixteen individually-programmable input/output pins (eight input, eight output, with enables to allow usage as eight input/output pins when combined with external tristate drivers)
- Control word read-back capability
- Additional test registers and modes implemented for functional verification and manufacturing test.

2.3 Example systems

Three example systems are provided with the ADK:

- *FRBM-based AMBA system* on page 2-6
- *Example AMBA system (ARM7TDMI-based EASY) design* on page 2-7
- *Example multi-layer AMBA system* on page 2-8.

2.3.1 FRBM-based AMBA system

This example provides an AMBA system based around the *File Reader Bus Master* (FRBM) that can be used to generate any AMBA transaction to exercise the bus and peripherals. The design is supplied with example files to drive the bus peripherals and can be used as the basis of a test bench for your peripherals. Figure 2-1 shows the FRBM-based example system.

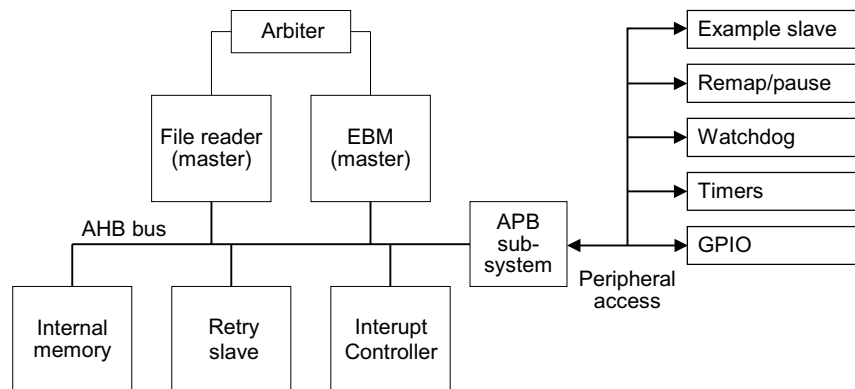


Figure 2-1 FRBM-based example system

2.3.2 Example AMBA system (ARM7TDMI-based EASY) design

This example provides a complete working example of an ARM7TDMI-based ASIC design. The design is supplied with:

- example software to run in the system
- synthesis scripts to generate a netlist version of the design
- examples of the use of a production test interface that allows the application of functional test vectors to the core.

Figure 2-2 shows the ARM7TDMI-based AMBA example system.

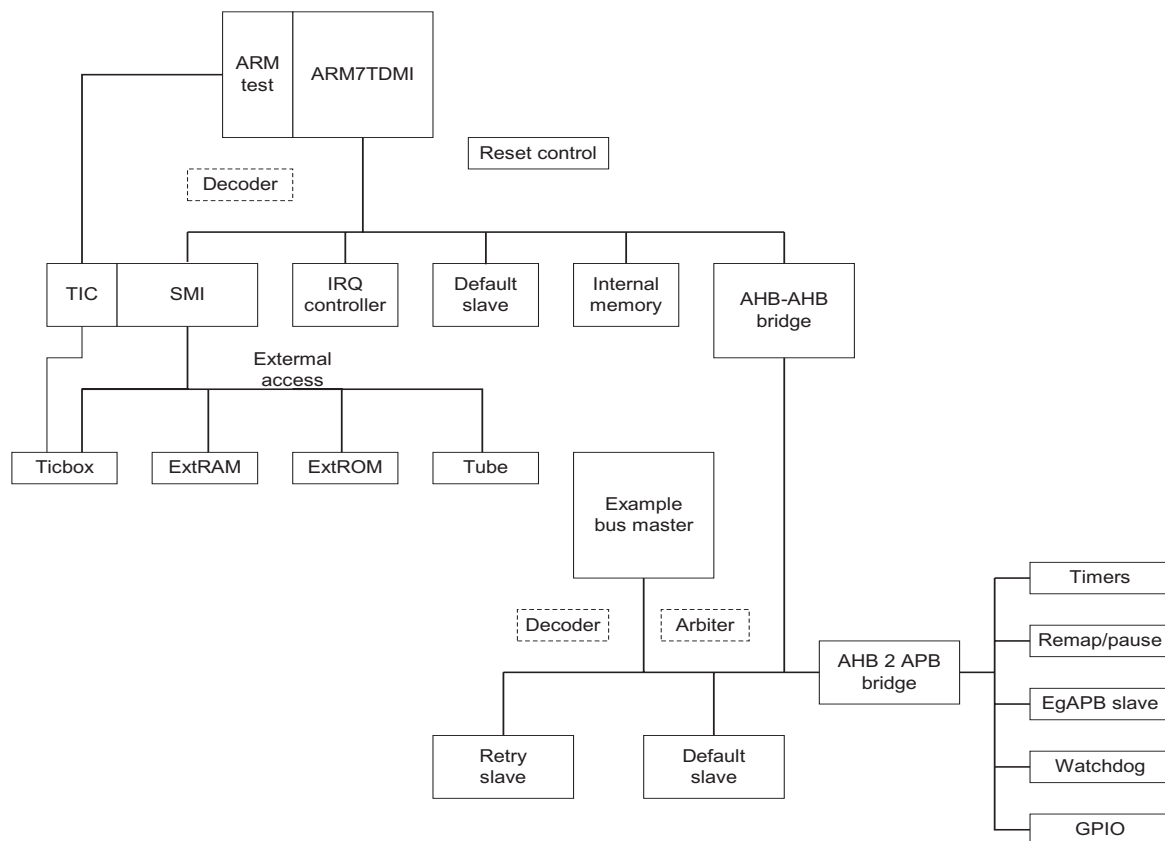


Figure 2-2 EASY-based example system

2.3.3 Example multi-layer AMBA system

This example provides a complete working example of an ARM922T-based ASIC design using the multi-layer AMBA architecture. The design is supplied with:

- example software to run in the system
- synthesis scripts to generate a netlist version of the design
- examples of the use of a production test interface that enables the application of functional test vectors to the core.

The system is configured with two layers (one with two masters on one AHB), a 2-to-3 interconnect matrix and three slave ports (one with three slaves sharing one port).

Figure 2-3 shows the multi-layer AMBA example system.

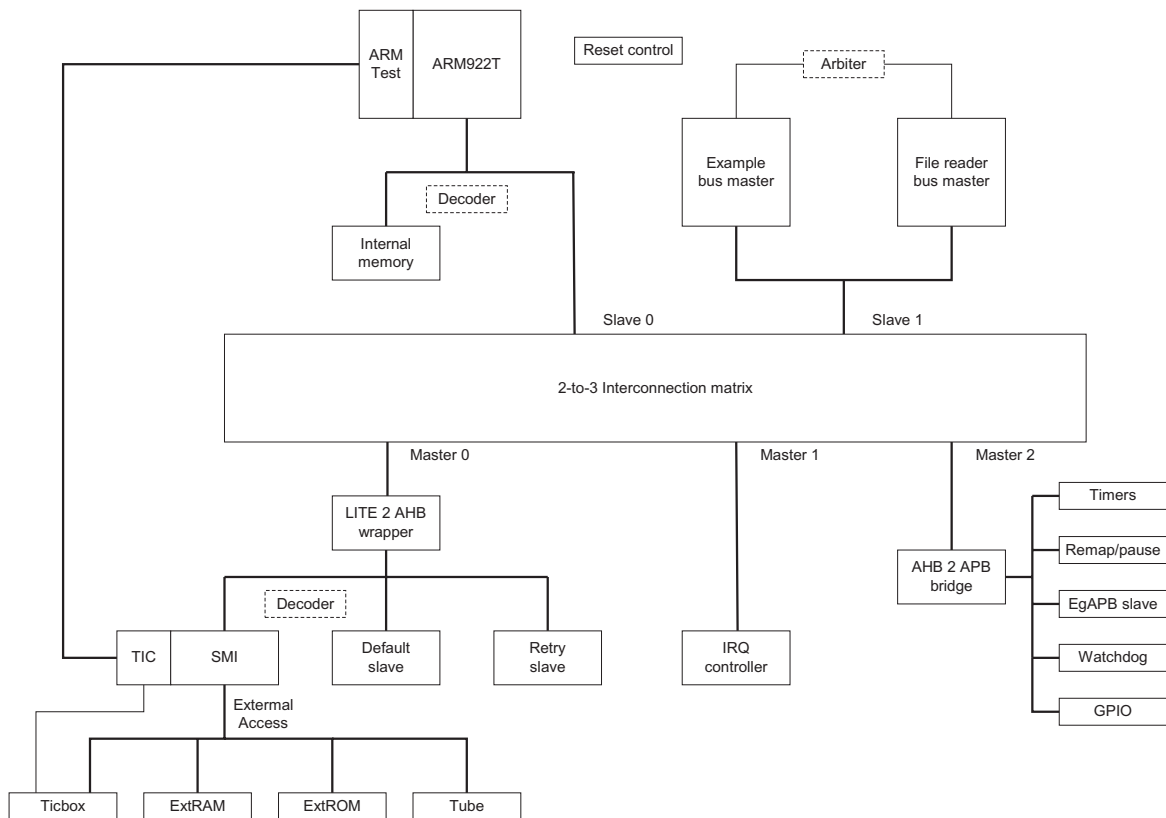


Figure 2-3 Multi-layer AMBA example system

Chapter 3

AHB Components

This chapter describes the AHB components used in the *AMBA Design Kit* (ADK). It contains the following sections:

- *Reset controller (ResetCntl)* on page 3-2
- *Arbiter (Arbiter3/Arbiter15)* on page 3-3
- *Default slave (DefaultSlave)* on page 3-4
- *Master to slave multiplexor (MuxM2S)* on page 3-5
- *Slave to master multiplexor (MuxS2M)* on page 3-6
- *Retry slave* on page 3-7
- *Static memory interface (SMI)* on page 3-8
- *Multi-layer bus switch (BusMatrix)* on page 3-17
- *Decoder (Decoder)* on page 3-19
- *APB bridge (APBif)* on page 3-22
- *Example bus master* on page 3-24
- *AHB to AHB bridge (Ahb2Ahb)* on page 3-25
- *AHB-Lite to AHB wrapper* on page 3-34
- *Interrupt controller (Interrupt)* on page 3-36.

3.1 Reset controller (*ResetCntl*)

The reset controller is used to generate the system reset signal from an external reset input. This module is based on a state machine, which is used to detect the external reset being asserted, and is used to generate the system reset output. Figure 3-1 shows the reset controller module block diagram.

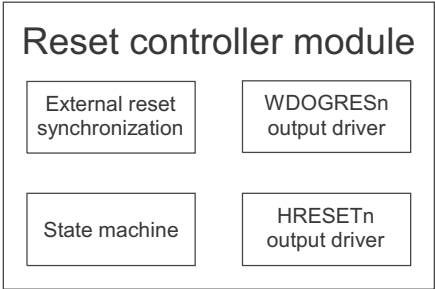


Figure 3-1 Reset controller module block diagram

3.1.1 Signal descriptions

Table 3-1 contains a list of non-AMBA signals used by the reset controller.

Table 3-1 Reset controller signal descriptions

Signal	Type	Direction	Description
nPOReset	Power-on reset	Input	Power-on reset input. This active LOW signal causes a cold reset when LOW. Can be asserted asynchronously to HCLK . The source of the nPOReset signal is implementation-dependent.
WDOGRES	Watchdog reset	Input	The watchdog clock domain reset input.
WDOGRESn	Watchdog reset	Output	The watchdog clock domain reset output.
HRESETn	System reset	Output	The system reset output.

3.2 Arbiter (*Arbiter3*)

The AMBA bus specification is a multi-master bus standard. As a result, a bus arbiter is required to ensure that only one bus master has access to the bus at any particular time. The arbiter can support up to three bus masters. Figure 3-2 shows the arbiter module block diagram.

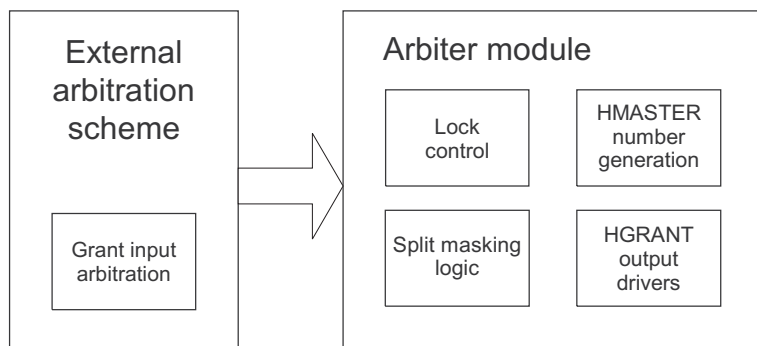


Figure 3-2 Arbiter module block diagram

3.2.1 Signal descriptions

Table 3-2 contains a list of non-AMBA signals used by the arbiter.

Table 3-2 Arbiter signal descriptions

Signal	Type	Direction	Description
HBUSREQMx	Bus request	Input	A signal from the bus master to the bus arbiter which indicates that the master requires the bus.
HLOCKMx	Locked transfers	Input	Lock signal from the bus master.
HGRANTMx	Bus grant	Output	This signal indicates that the bus master is currently the highest priority master. Ownership of the address/control signals changes at the end of a transfer when HREADY is HIGH, so the master gets access to the bus when both HREADY and HGRANTMx are HIGH.
HMASTERD [3:0]	Bus master	Output	Indicates which bus master owns the current data phase. Used by MuxM2S to remove the requirement for sequential logic within that block.

Note

For a description of the AMBA signals used by the arbiter, see *AMBA signals* on page 1-4.

3.3 Default slave (*DefaultSlave*)

The default slave is used to respond to transfers that are made to undefined regions of memory, where no AHB system slaves are mapped. A zero wait OKAY response is made to IDLE or BUSY transfers, with an ERROR response being generated if a NONSEQUENTIAL or SEQUENTIAL transfer is performed. Figure 3-3 shows the default slave module block diagram.

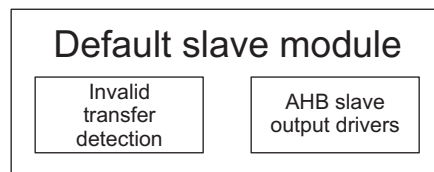


Figure 3-3 Default slave module block diagram

3.3.1 Signal descriptions

The default slave uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

3.4 Master to slave multiplexor (*MuxM2S*)

The master to slave multiplexor is used to connect the outputs of each AHB master to all of the AHB slaves on the bus segment. It uses the values on **HMASTER** and **HMASTERD** to select the appropriate bus master outputs, and also generates the default master outputs when no other masters are selected. The default configuration is three masters plus one dummy master. Figure 3-4 shows the master to slave module block diagram.

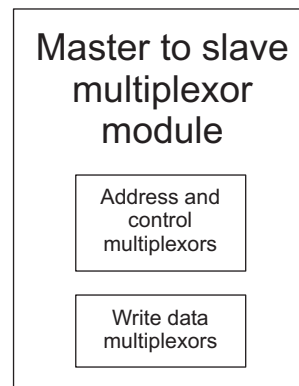


Figure 3-4 Master to slave multiplexor module block diagram

3.4.1 Signal descriptions

The master to slave multiplexor uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

3.5 Slave to master multiplexor (*MuxS2M*)

The slave to master multiplexor is used to connect the read data and response signals of the system bus slaves to the bus masters. It uses the current decoder **HSELx** outputs to select the bus slave outputs to use. The default configuration is for seven slaves. Figure 3-5 shows the slave to master module block diagram.

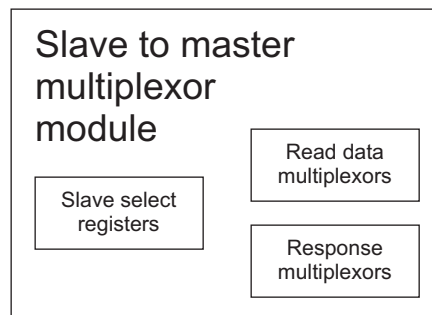


Figure 3-5 Slave to master multiplexor module block diagram

3.5.1 Signal descriptions

The slave to master multiplexor uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

3.6 Retry slave

The retry slave is a rudimentary module that is used to demonstrate how to build an AHB slave. The example contains very little functionality and consists of four 32-bit wide registers. The slave generates various logic functions of these registers, which can be read from different locations. Figure 3-6 shows a basic block diagram of the retry slave module system.

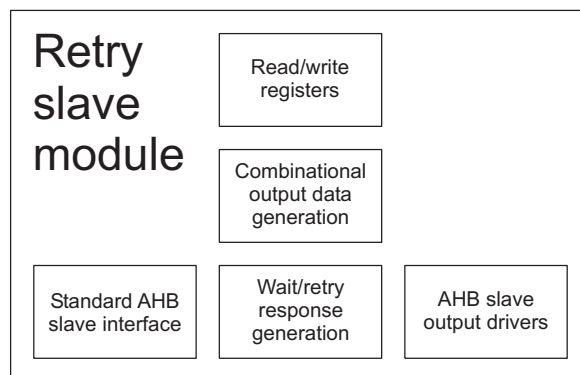


Figure 3-6 Retry slave module block diagram

3.6.1 Signal descriptions

The retry slave uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

3.7 Static memory interface (SMM)

The *Static Memory Interface* (SMI) is an AMBA-compliant example design which shows the basic requirements of an *External Bus Interface* (EBI). It is not intended to be a ready-made EBI for a real system because such an EBI design has to take process, package, and varying external delays into account.

The SMI is an AMBA slave module, and connects to the AHB. It contains an AHB *Test Interface Controller* (TIC) AMBA master block that can be used to test the processor core using externally applied TIC vectors.

The SMI can be used to provide an interface between an AMBA AHB system bus and external (off-chip) memory devices. The SMI provides support for up to eight independently configurable memory banks simultaneously. Each memory bank is capable of supporting:

- SRAM
- ROM.

A block diagram of the SMI is shown in Figure 3-7.

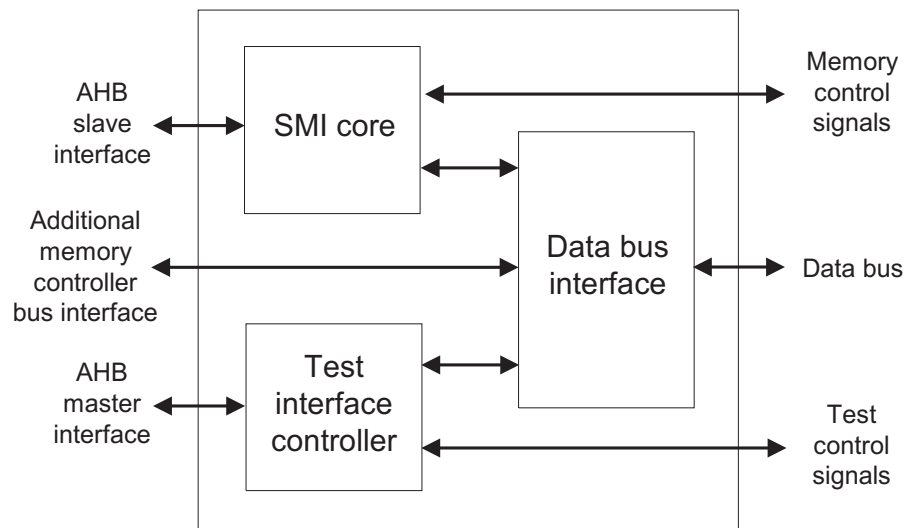


Figure 3-7 SMI block diagram

Test interface controller

The TIC is used during testing to read external test vectors and apply them to the system through the AMBA AHB master interface.

3.7.1 SMI programmer's model

Eight memory banks are supported, with a separate chip select output for each bank. The chip select lines **SMCS[7:0]** for all banks are configurable to be either active HIGH or active LOW (default). The memory bank selection is controlled by the AMBA AHB address lines **HADDR[28:26]**, as shown in Table 3-3, where all **SMCS** are shown as active HIGH.

Table 3-3 Static memory bank select coding (Remap = 1)

HADDR [28:26]	SMCS[7:0]	Memory bank (Remap 1)	Memory bank (Remap 0)
000	00000001	Bank 0	Bank 7
001	00000010	Bank 1	Bank 1
010	00000100	Bank 2	Bank 2
011	00001000	Bank 3	Bank 3
100	00010000	Bank 4	Bank 4
101	00100000	Bank 5	Bank 5
110	01000000	Bank 6	Bank 6
111	10000000	Bank 7	Bank 7

The base address of the external memory banks and the base address of the SMI memory bank registers are defined in the AMBA AHB address decoder, that is used to generate the AHB slave select signal **HSEL_{SMC}**.

3.7.2 Test interface controller

The *Test Interface Controller* (TIC) is a state machine that provides an AMBA AHB bus master for system test. It reads test write and address data from the external data bus **SMDATA**, and drives the external bus with test read data, allowing the use of only one set of output tristate buffers onto **SMDATA**.

The TIC is used to convert externally applied test vectors into internal transfers on the AHB bus. A three-wire external handshake protocol is used, with two inputs controlling the type of vector that is applied and a single output that indicates when the next vector can be applied.

Typically the TIC is the highest priority AMBA bus master, which ensures test access under all conditions.

The TIC model supports address incrementing and control vectors. This means that the address for burst transfers can automatically be generated by the TIC.

3.7.3 TIC programmer’s model

The TIC operates as a standard AHB bus master during system test when the external test pins show that the system is required to enter test mode. In this mode, the requests control of the AHB, and when granted uses the AHB to perform system tests.

Table 3-4 shows the operation of the external test pins to change the TIC mode from normal operation into test mode.

Table 3-4 Test control signals during normal operation

TESTREQA	TESTREQB	TESTACK	Description
0	-	0	Normal operation
1	-	0	Enter test mode request
-	-	1	Test mode entered

During system test the external test pins are used to control the operation of the TIC. The operation of these pins is shown in Table 3-5.

Table 3-5 Test control signals during test operation

TESTREQA	TESTREQB	TESTACK	Description
-	-	0	Current access incomplete
1	1	1	Address vector or Control vector or Turnaround vector
1	0	1	Write vector
0	1	1	Read vector
0	0	1	Exit test mode

On entry into test mode the TIC indicates that it has switched to the test clock input by asserting the **TESTACK** signal.

Test vector types

There are five types of test vector associated with the test interface:

Address vector	The address for all subsequent read and write transfers is sampled by the TIC.
Write vector	The TIC performs an AHB write cycle, using the write data currently driven onto the external data bus.
Read vector	The TIC performs an AHB read cycle, driving the read data onto the external data bus when it becomes valid.
Control vector	Internal TIC registers are set, that control the types of read and write transfers that are performed.
Turnaround vector	Used between a read cycle and a write cycle to avoid clashes on the external data bus.

The address, control and turnaround vectors are all indicated by the same value on the **TESTREQA** and **TESTREQB** signals. You can use the following rules to determine which type of vector is being applied:

- a read vector, or burst of read vectors, is followed by two turnaround vectors
- when a single address or control vector is applied it is an address vector
- when multiple address or control vectors are applied they are all address vectors, apart from the last which is a control vector.

Control vectors

The control vector is used to determine the types of transfer the TIC can perform, by setting the values of the **HSIZETIC**, **HPROTTIC**, and **HLOCKTIC** AHB master outputs.

The default TIC bus master transfer type is:

32-bit transfer width

HSIZETIC[2:0] signifies word transfer.

Privileged system access

HPROTTIC[3:0] signifies supervisor data access, uncacheable and unbufferable.

Bit 0 of the control vector is used to indicate if the control vector is valid. Therefore if a control vector is applied with bit 0 LOW, the vector is ignored and does not update the control information. This mechanism allows address vectors that have bit 0 LOW to be applied for many cycles without updating the control information.

Although the default settings are sufficient for testing many systems, control vectors can be used to change the control signals of the transfer, and can also be used to select whether the TIC must generate fixed addresses or incrementing addresses.

Table 3-6 defines the bit positions of the control vector. The control vector bit definitions are designed to be backwards compatible with earlier versions of the TIC and therefore not all of the control bits are in obvious positions.

Table 3-6 Control vector bit definitions

Bit position	Description
0	Control vector valid
1	Reserved
2	HSIZETIC[0]
3	HSIZETIC[1]
4	HLOCKTIC
5	HPROTTIC[0]
6	HPROTTIC[1]
7	Address increment enable
8	Reserved
9	HPROTTIC[2]
10	HPROTTIC[3]

There is no mechanism to control the types of burst that the TIC can perform and only incrementing bursts of an undefined length are supported. The TIC only supports 8-bit, 16-bit, and 32-bit transfers and therefore **HSIZETIC[2]** cannot be altered and is always LOW.

To support burst accesses using the test interface the TIC can support incrementing of the bus address. The TIC increments eight address bits and the address range that can be covered by this incrementer depends on the size of the transfers being performed.

The control vector provides a mechanism to enable and disable the address incrementer within the TIC. This allows burst accesses to incremental addresses, as would be used for testing internal RAM. Alternatively the address increment can be disabled, so that successive accesses of a burst occur to the same address, as would be required to continually read from a single peripheral register.

The address incrementer is disabled by default and must be enabled using a control vector prior to use.

Note

The control vector is primarily used to change signals that have the same timing as the address bus. However the control vector also allows the lock signal to be changed, which is actually required before the locked transfer commences. If the **HLOCKTIC** signal is used during testing it must be set a transfer before it is required. This difference in timing on the **HLOCKTIC** signal can in some cases cause an additional transfer to be locked both before and after the sequence that must in fact be locked.

3.7.4 Signal descriptions

Table 3-7 contains a list of non-AMBA signals used by the SMI. A number of pins, although present, are not used on the SMI, but are included for pin compatibility with the PrimeCell SMC (PL092). These are indicated where applicable.

Table 3-7 Signal descriptions

Signal	Type	Direction	Description
nHCLK	Input	Clock control	Not used on SMI.
HSELSMC	Input	AMBA AHB decoder	Slave select signal for PrimeCell AHB SMI memory banks.
HSELREG	Input	AMBA AHB decoder	Not used on SMI.
HRESPTIC[1:0]	Input	AMBA AHB slave	The transfer response provides additional information on the status of a transfer. The TIC supports both SPLIT and RETRY responses.
HRDATATIC[31:0]	Input	AMBA AHB slave	The read data bus is used to transfer data from bus slaves to the bus master during test read operations.
HGRANTTIC	Input	AMBA AHB arbiter	This signal indicates that the TIC is currently the highest priority master. Ownership of the address or control signals changes at the end of the transfer when HREADYIN is HIGH.
HADDRTIC[31:0]	Output	AMBA AHB slave	The 32-bit system address bus.
HTRANSTIC[1:0]	Output	AMBA AHB slave	Indicates the type of the current transfer, which can be NONSEQUENTIAL, SEQUENTIAL, or IDLE. The TIC does not use the BUSY transfer type.

Table 3-7 Signal descriptions (continued)

Signal	Type	Direction	Description
HWRITETIC	Output	AMBA AHB slave	Transfer direction signal. When HIGH, this signal indicates a write to the SMI and when LOW, a read from the SMI.
HSIZETIC[2:0]	Output	AMBA AHB slave	Transfer size signal. This signal indicates the size of the current transfer, which can be byte (8-bit), halfword (16-bit) or word (32-bit). The TIC does not support larger transfer sizes.
HREADYINTIC	Input	Other AHB TIC slaves	Transfer completed input. Multiplexed HREADY input from all slaves on TIC AHB bus.
HBURSTTIC[2:0]	Output	AMBA AHB slave	Indicates if the transfer forms part of a burst. The TIC always performs incrementing bursts of unspecified length.
HPROTTIC[3:0]	Output	AMBA AHB slave	The protection control signals indicate if the transfer is an opcode fetch or data access, as well as if the transfer is a Supervisor mode access or User mode access. These signals can also indicate whether the current access is cacheable or unbufferable.
HWDATATIC[31:0]	Output	AMBA AHB slave	The write data bus is used to transfer data from the master to bus slaves during write operations. A minimum data bus width of 32 bits is recommended, however this can easily be extended to allow for higher bandwidth operation.
HBUSREQTIC	Output	AMBA AHB arbiter	A signal from the TIC to the bus arbiter that indicates that it requires the bus.
HLOCKTIC	Output	AMBA AHB arbiter	When HIGH this signal indicates that the TIC requires locked access to the bus and no other master must be granted the bus until this signal is LOW.
BIGENDIAN	Input	System	Not used on SMI.
EXTBUSMUX	Input	System	Not used on SMI.
MCBUSREQ	Input	Additional memory controller	Not used on SMI.
MCADDR[31:0]	Input	Additional memory controller	Not used on SMI.
MCDATAOUT[31:0]	Input	Additional memory controller	Not used on SMI.

Table 3-7 Signal descriptions (continued)

Signal	Type	Direction	Description
MCDATAEN[3:0]	Input	Additional memory controller	Not used on SMI.
MCBUSGNT	Output	Additional memory controller	Not used on SMI.
SMBUSREQEBI	Output	External bus multiplexer	Not used on SMI.
SMBUSGNTEBI	Input	External bus multiplexer	Not used on SMI.
TICBUSREQEBI	Output	External bus multiplexer	Not used on SMI.
TICBUSGNTEBI	Input	External bus multiplexer	Not used on SMI.
TICREADEBI	Output	External bus multiplexer	Not used on SMI.
TBUSOUTEBI[31:0]	Output	External bus multiplexer	Not used on SMI.
REMAP	Input	System	Indicates the state of the memory map: 0 = reset memory map (SMCS7 mapped to SMCS0) 1 = normal memory map.
SCANENABLE	Input	System	Dummy pin that is used as a dedicated scan enable input.
SCANINnHCLK	Input	System	Not used on SMI.
SCANINHCLK	Input	System	Dummy pin that is used as a dedicated HCLK scan chain input.
SCANOUTnHCLK	Output	System	Not used on SMI.
SCANOUTHCLK	Output	System	Dummy pin that is used as a dedicated HCLK scan chain input.
SMMWCS7[1:0]	Input	Input pad	Not used on SMI.
SMWAIT	Input	Input pad	Not used on SMI.
CANCELSMWAIT	Input	Input pad	Not used on SMI.
SMDATAIN[31:0]	Input	Input pad	External input data bus used to read data from memory bank.
SMDATAOUT[31:0]	Input	Input pad	External output data used to write data from SMI to memory bank.

Table 3-7 Signal descriptions (continued)

Signal	Type	Direction	Description
SMADDR[25:0]	Output	Output pad	External memory address bus, to external memory banks.
SMCS[7:0]	Output	Output pad	Chip select for external memory banks 7 to 0, default active LOW.
nSMDATAEN[3:0]	Output	Output pad	Tristate input/output pad enable for the byte lanes of the external memory data bus SMDATA[31:0] , active LOW. Enables the byte lanes [31:24], [23:16], [15:8], and [7:0] of the data bus independently.
nSMWEN	Output	Output pad	Not used on SMI.
nSMBLS[3:0]	Output	Output pad	Byte lane select signals, active LOW. The signals nSMBLS[3:0] select byte lanes [31:24], [23:16], [15:8], [7:0] on the data bus.
nSMOEN	Output	Output pad	Output enable for external memory banks, active LOW.
TESTREQA	Input	Input pad	This is the Test Bus Request A input signal and is required as a dedicated device pin. During normal system operation the TESTREQA signal is used to request entry into the test mode. During test TESTREQA is used, in combination with TESTREQB , to indicate the type of test vector that is applied in the following cycle.
TESTREQB	Input	Input pad	During test this signal is used, in combination with TESTREQA , to indicate the type of test vector that is to be applied in the following cycle.
TESTACK	Output	Output pad	The test bus acknowledge signal gives external indication that the TIC is granted and also indicates when a test access is complete. When TESTACK is LOW, the current test vector must be extended until TESTACK becomes HIGH.

———— **Note** ————

For a description of the AMBA signals used by the SMI, see *AMBA signals* on page 1-4.

3.8 Multi-layer bus switch (*BusMatrix*)

Multi-layer AHB is an interconnection scheme, based on the AHB protocol, that has the additional advantage of enabling parallel access paths between multiple masters and slaves in a system. This is achieved by using a more complex interconnection matrix and gives the benefit of increased overall bus bandwidth as well as a more flexible system architecture. Figure 3-8 shows the multi-layer bus switch block diagram.

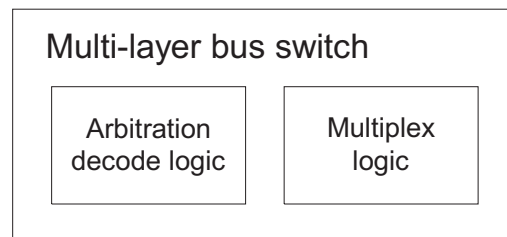


Figure 3-8 Multi-layer bus switch block diagram

3.8.1 Programmer's model

The following modifications to the bus switch are possible, but are not supported within the RTL:

- different address bus widths
- different data bus widths
- removal of **HPROT** signals.
- removal of **HBURST** signals.
- addition of **HMASTER** signals for split transfer support.

3.8.2 Signal descriptions

Table 3-8 contains a list of non-AMBA signals used by the bus switch. An identical set of signals can be found on each of the input ports and each of the slave output ports.

Table 3-8 AHB signal description for multi-layer bus switch

Name	Direction	Description
Input port signals		
HSELSx	Input	Input port x select signal
HADDRSx[31:0]	Input	Input port x address

Table 3-8 AHB signal description for multi-layer bus switch (continued)

Name	Direction	Description
HTRANSx[1:0]	Input	Input port x transfer type
HWRITESx	Input	Input port x transfer direction
HSIZESx[2:0]	Input	Input port x transfer size
HBURSTx[2:0]	Input	Input port x burst type
HPROTSx[3:0]	Input	Input port x protection control
HWDATASx[31:0]	Input	Input port x write data
HMASTLOCKSx	Input	Input port x locked transfer
HREADYx	Input	Input port x ready signal
HRDATASx[31:0]	Output	Input port x read data
HREADYOUTSx	Output	Input port x ready output
HRESPSx[1:0]	Output	Input port x response
Output port signals		
HSELMx	Output	Output port x select signal
HADDRMx[31:0]	Output	Output port x address
HTRANSMx[1:0]	Output	Output port x transfer type
HWRITEMx	Output	Output port x transfer direction
HSIZEMx[2:0]	Output	Output port x transfer size
HBURSTMx[2:0]	Output	Output port x burst type
HPROTMx[3:0]	Output	Output port x protection control
HWDATAMx[31:0]	Output	Output port x write data
HMASTLOCKMx	Output	Output port x locked transfer
HREADYMx	Output	Output port x ready signal
HRDATAMx[31:0]	Input	Output port x read data
HREADYOUTMx	Input	Output port x ready output
HRESPMx[1:0]	Input	Output port x response

3.9 Decoder (*Decoder*)

The system decoder is used to decode the address bus and generate select lines to each of the system bus slaves, indicating that a read or write access to that slave is required. The default configuration is sixteen slots. Figure 3-9 shows the decoder module block diagram.

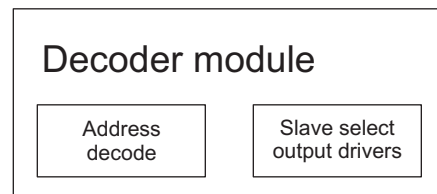


Figure 3-9 Decoder module block diagram

3.9.1 Programmer's model

This section of code is used to define the memory map for the whole system. If modules are added, removed, or moved to new locations, the code must be modified to match these system changes, ensuring that the correct slave is selected for each address used.

The decoder controls the memory map of the system, and generates a slave select signal for each memory region. The **Remap** signal is used to provide a different memory map at reset, when ROM is required at address `0x00000000`, and during normal operation, when internal RAM can be used at address `0x00000000`.

The **Remap** signal is typically provided by a remap and pause peripheral, that drives **Remap** LOW at reset. The signal is driven HIGH only after a particular address in the remap and pause peripheral is accessed.

Figure 3-10 shows both the normal and reset memory maps.

Address	Normal memory map	Reset memory map
0xFFFFFFFF	Interrupt controller	Interrupt controller
0xF0000000	Unused	Unused
0xE0000000	Retry slave	Retry slave
0xD0000000	APB peripherals	APB peripherals
0xC0000000	Unused	Unused
0x80000000	Internal memory	Internal memory
0x70000000	Unused	Unused
0x40000000	External static memory	External static memory
0x30000000	Unused	Unused
0x10000000	Unused (reserved for SDRAM)	Unused (reserved for SDRAM)
0x00100000	Internal memory alias	External static memory alias
0x00000000		

Figure 3-10 System memory map

3.9.2 Signal descriptions

Table 3-9 contains a list of non-AMBA signals used by the decoder module.

Table 3-9 Decoder module signal descriptions

Signal	Type	Direction	Description
Remap	Reset memory map	Input	When LOW, the internal memory is not part of the system memory map, and external memory is mapped from address 0x00000000 that contains the system startup code. In normal operation this signal is HIGH, allowing use of the internal memory.

———— **Note** ————

For a description of the AMBA signals used by the decoder, see *AMBA signals* on page 1-4.

3.10 APB bridge (APBif)

The AHB-APB bridge is an AHB slave, providing an interface between the high- speed AHB domain and the low-power APB domain. Read and write transfers on the AHB are converted into corresponding transfers on the APB through a master interface. Because the APB is not pipelined, wait states are added during transfers to and from the APB when the AHB is required to wait for the APB protocol. The default configuration is sixteen slots. Figure 3-11 shows the AHB-APB bridge module block diagram.

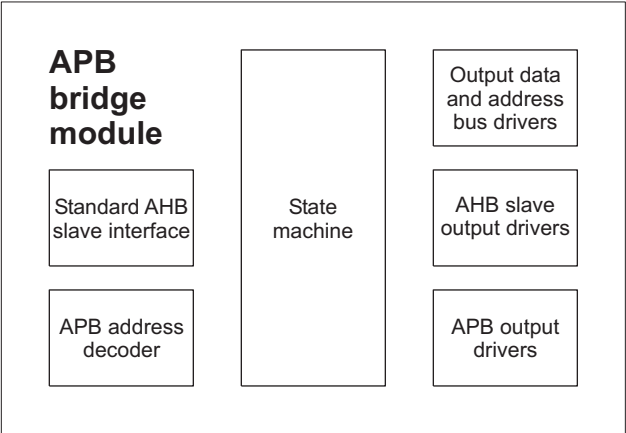


Figure 3-11 AHB-APB bridge module block diagram

3.10.1 Programmer's model

The AHB-APB bridge controls the memory map for the peripherals, and generates a select signal for each peripheral. Figure 3-12 shows the default system memory map.

Address	Peripheral memory map
0xD0000000	Example APB slave
0xCF000000	
	Unused
0xC9000000	Remap and pause
0xC8000000	
	Unused (reserved for extra GPIO)
0xC5000000	GPIO
0xC4000000	
	Unused (reserved for timers)
0xC3000000	Timers
0xC2000000	
	Watchdog
0xC1000000	Unused (reserved for system controller)
0xC0000000	

Figure 3-12 Peripheral memory map

————— **Note** —————

Timing diagrams showing the relationship between AHB and APB transfers can be found in the *APB Specification*.

3.10.2 Signal descriptions

The AHB-APB bridge uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

————— **Note** —————

Timing diagrams showing the relationship between AHB and APB transfers can be found in the *APB Specification*.

3.11 Example bus master

The *Example Bus Master* (EBM) consists of:

- an AHB-Lite core, which generates a fixed set of transfers in a continuous fashion
- an AHB-Lite to AHB wrapper, which enables its connection to a standard AHB bus.

This is intended to provide an example of both AHB-Lite bus master design and the use of the wrapper to interface to an AHB system. It can also be used as a simple way to generate AHB transactions in order to congest the bus for validation purposes.

3.11.1 Programmer's model

The EBM has a fixed functionality and is not programmable. There are however, simulation-time options (using VHDL generics or Verilog parameters):

EBMenable	when LOW prevents the master from generating any transfers.
EBMreadAddr	the base address for the read burst (bits [31:24] only).
EBMwriteAddr	the address for write transfers (bits [31:24] only).
EBMinitCount	the number of IDLE transactions between bursts (0 to 1023).

3.12 AHB to AHB bridge (*Ahb2Ahb*)

The AHB-AHB bridge provides a unidirectional link between two AHB domains. Its function is to allow a master to access a slave on another bus, with the transfer initiated from one side only. Figure 3-13 shows the AHB-AHB bridge module in a typical system.

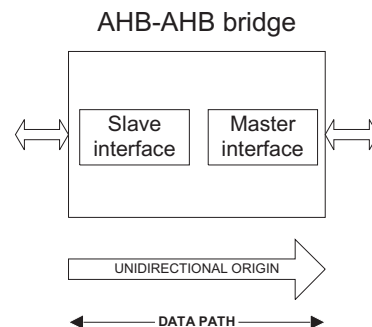


Figure 3-13 AHB-AHB bridge in a typical system

3.12.1 Features

The AHB-AHB bridge includes the following features:

- A system bus clock that is identical and synchronous across all AHB domains.
- A 32-bit data bus (endian independent) and a 32-bit address bus in both AHB systems.
- Registered paths for timing isolation, concerning:
 - Data bus in both directions.
 - Address bus.
 - Control bus.
 - **HREADY** signal.
- Address generation to reduce protocol overhead, using the information provided on the bridge slave interface at the start of a transfer.
- One-to-one address mapping, that is the memory maps of both systems do not overlap.
- All burst types are supported, single, incrementing, and wrapping.
- Single-cycle write with a one-stage buffer, active when **HPROT[2]** is HIGH for the transfer. Non-buffered writes take three cycles.
- 2-cycle read with BUSY insertion.
- LOCKed transfers are permitted through the bridge.
- Burst transfers rebuilt in response to a SPLIT or RETRY.
- SPLIT and RETRY responses from remote slaves are supported, but they are not propagated back to the master. The bridge will insert wait states on bus 1 using **HREADY** to hold up the master until the remote slave is ready to complete the outstanding transfer. The bridge slave interface does not support SPLIT or RETRY responses.
- ERROR responses from remote slaves are normally propagated back to the master to determine further action for that transfer. This response is suppressed for buffered writes because of a transfer correlation issue.

Note

The initial transfer(s) of a burst and transfers of type SINGLE take longer to complete.

3.12.2 Programmer's model

In the example system shown in Figure 3-13 on page 3-25, the general concept of the bridge from a programmer's viewpoint is shown in the following sections:

- *Preserved address map* on page 3-27
- *Aliased or piecewise address map* on page 3-28
- *Signal paths* on page 3-29.

Note

The scope of memory-map visibility is user-defined.

Preserved address map

The memory maps for a system with one-to-one address mapping are shown in Figure 3-14. It is assumed that full visibility of the slaves in system 2 is required.

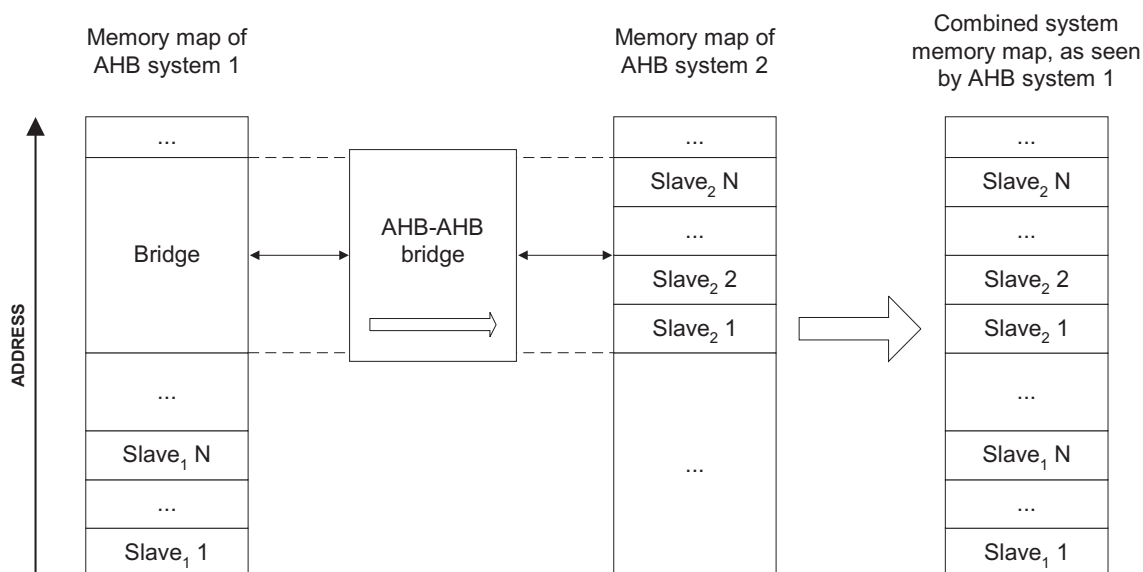


Figure 3-14 System memory maps without address aliasing

Aliased or piecewise address map

Address-aliasing is a function of the address decoder and of masking-off the high-order address bits in hardware. It is recommended that this is implemented in AHB system 1. A generalized scheme is shown in Figure 3-15.

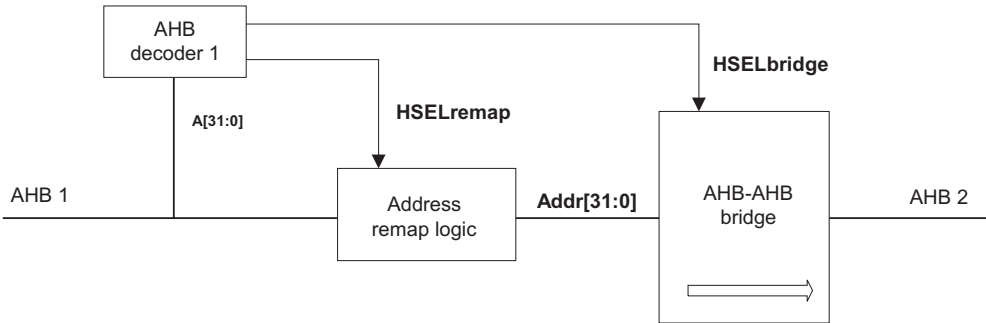


Figure 3-15 Address-aliasing hardware

Figure 3-16 and Figure 3-17 on page 3-29 show the use of address-aliasing to make memory in system 2 appear both at address 0 and also at a higher address, so that it can be accessed by system 1 through the bridge. In this design, only the area of ROM is visible through the bridge.

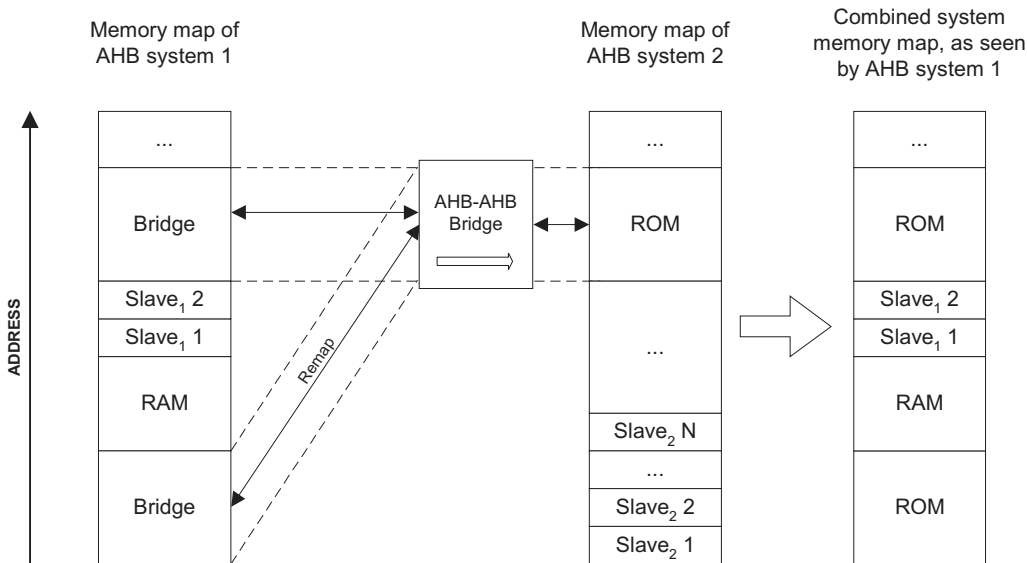


Figure 3-16 System memory maps with aliased addressing

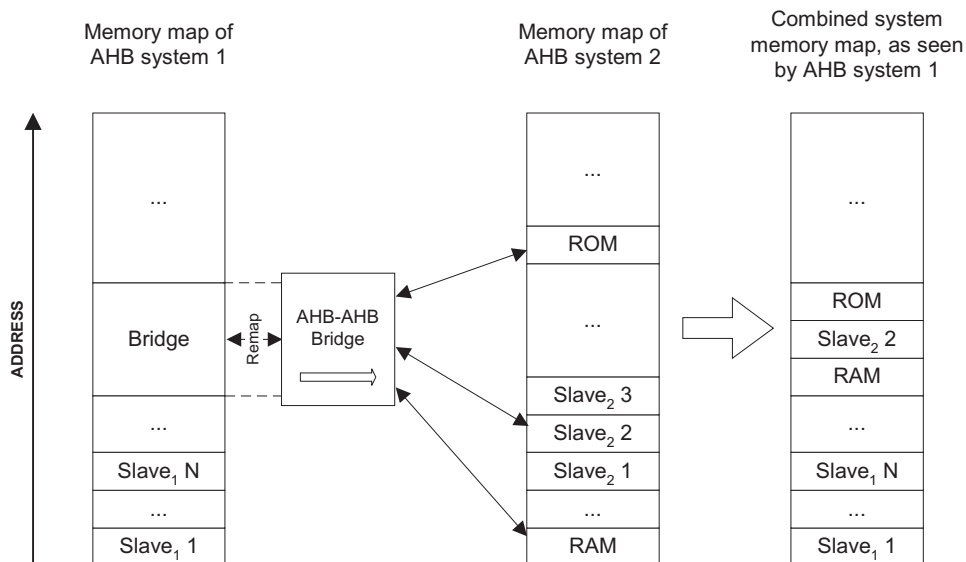


Figure 3-17 System memory maps with piecewise addressing

Signal paths

The bridge passes the value on **HPROTS[3:0]** from master to remote slave without modification. The value on **HTRANS[1:0]** is passed through the bridge, but is appropriately modified by the master interface when a burst is being rebuilt, when the bridge is granted but idle and when BUSY wait-states are required. The value on **HBURSTS[2:0]** is passed through the bridge, except when the master interface is rebuilding a burst. The data bus is registered in both directions. The address bus, control bus, **HMASTLOCKS**, and **HREADYOUTS** are all registered.

3.12.3 Signal descriptions

Table 3-10 contains a list of the signals used by the AHB-AHB bridge.

Table 3-10 AHB-AHB bridge interface signals

Signal	Block	AHB bus	Direction	Description
HSELS	Slave I/F	1	Input	The bridge slave interface uses the HSELS signal to determine when it must respond to a bus transfer.
HADDRS[31:0]	Slave I/F	1	Input	The 32-bit system address bus.
HTRANS[1:0]	Slave I/F	1	Input	Indicates the type of the current transfer, which can be NONSEQUENTIAL, SEQUENTIAL, IDLE, or BUSY.
HWRITES	Slave I/F	1	Input	When HIGH, this signal indicates a write transfer, and when LOW, it indicates a read transfer.
HSIZES[2:0]	Slave I/F	1	Input	Indicates the size of the transfer. The bridge uses 32-bit data for read and write transfers.
HBURSTS[2:0]	Slave I/F	1	Input	Indicates if the transfer forms part of a burst. The bridge supports all types of transfer (single, incrementing, or wrapping).
HPROTS[3:0]	Slave I/F	1	Input	The protection control signals provide additional information about a bus access and are primarily intended for use by any module that has to implement some level of protection.
HWDATAS[31:0]	Slave I/F	1	Input	The 32-bit write data bus is used to transfer data from the bus master to the bridge during a write operation.
HRDATAS[31:0]	Slave I/F	1	Output	The 32-bit read data bus is used to transfer data from the bridge to the bus master during a read operation.
HRESPS[1:0]	Slave I/F	1	Output	The transfer response provides additional information on the status of a transfer. Only two responses are supported on the slave interface, OKAY and ERROR.
HREADYOUTS	Slave I/F	1	Output	When HIGH, the HREADYOUTS signal indicates that a transfer has finished on bus 1. This signal can be driven LOW by the bridge to extend a transfer.
HREADYS	Slave I/F	1	Input	Input version of HREADYOUTS , required by the slave interface.

Table 3-10 AHB-AHB bridge interface signals (continued)

Signal	Block	AHB bus	Direction	Description
HMASTLOCKS	Slave I/F	1	Input	When HIGH, this signal indicates that the master on bus 1 requires locked access through the bridge and no other master must be granted the bus until this signal is LOW.
HBUSREQM	Master I/F	2	Output	A signal from the bridge to the arbiter, that indicates that the master interface requires bus 2. There is an HBUSREQ signal for each bus master in the system.
HLOCKM	Master I/F	2	Output	When HIGH, this signal indicates that the master requires locked access on bus 2 and no other master must be granted that bus until this signal is LOW.
HGRANTM	Master I/F	2	Input	This signal indicates that the bridge is currently the highest priority master. Ownership of the address/control signals changes at the end of a transfer when HREADYM is HIGH, so the master gets access to the bus when both HREADYM and HGRANTM are HIGH.
HADDRM[31:0]	Master I/F	2	Output	The 32-bit system address bus.
HTRANSM[1:0]	Master I/F	2	Output	Indicates the type of the current transfer, which can be NONSEQUENTIAL, SEQUENTIAL, IDLE, or BUSY.
HWRITEM	Master I/F	2	Output	When HIGH, this signal indicates a write transfer, and when LOW, it indicates a read transfer.
HSIZEM[2:0]	Master I/F	2	Output	Indicates the size of the transfer. The bridge uses 32-bit data for read and write transfers.
HBURSTM[2:0]	Master I/F	2	Output	Indicates if the transfer forms part of a burst. The bridge supports all types of transfer (single, incrementing, or wrapping).
HPROTM[3:0]	Master I/F	2	Output	The protection control signals provide additional information about a bus access and are primarily intended for use by any module that has to implement some level of protection.
HWDATAM[31:0]	Master I/F	2	Output	The 32-bit write data bus is used to transfer data from the bridge to the slave(s) on bus 2, during write operations.

Table 3-10 AHB-AHB bridge interface signals (continued)

Signal	Block	AHB bus	Direction	Description
HRDATAM[31:0]	Master I/F	2	Input	The 32-bit read data bus is used to transfer data from the slave(s) on bus 2, to the bridge, during read operations.
HRESPM[1:0]	Master I/F	2	Input	The transfer response provides additional information on the status of a transfer. Four different responses are supported, OKAY, ERROR, RETRY, and SPLIT.
HREADYM	Master I/F	2	Input	When HIGH, the HREADYM signal indicates that a transfer has finished on bus 2. This signal can be driven LOW by a slave to extend a transfer.

———— **Note** ————

For a description of the AMBA signals used by the AHB-AHB bridge, see *AMBA signals* on page 1-4.

———— **Note** ————

The **HMASTLOCKS** signal is normally associated with a SPLIT-capable slave, but the bridge slave interface does not support that feature. The signal is required in this design because the bridge supports LOCKed transfers.

3.12.4 ERROR, SPLIT, and RETRY responses

The ERROR response from a slave is normally passed back to the controlling master on bus 1, which determines further action for that transfer. If a slave gives an ERROR response, the **MERROR** signal is asserted from a combinatorial decode of the **HRESPM[1:0]** bus. The slave interface re-encodes that line to drive a registered ERROR response on **HRESPS[1:0]**. The master on bus 1 can meaningfully interpret an ERROR response when the transfer is either a read or a non-buffered write. However, the ERROR response is overwritten with OKAY for buffered writes, because it cannot be correlated with the original transfer.

To verify completion of a buffered-write transfer, you can perform a SINGLE or INCR read immediately after the transfer. If the read operation completes, the buffered-write is completed.

The SPLIT and RETRY responses on **HRESPM[1:0]** are overwritten with OKAY on the slave interface, and translated accordingly into wait-states using **HREADYOUTS**.

3.12.5 Pipeline overhead

Because a number of signal paths are registered, the bridge master interface can insert BUSY wait states appropriately during a transfer. The burst-read transaction takes a minimum of 2 clock cycles, consisting of the data with at least one wait state per unit transfer. For the burst-write transaction, the number of clock cycles depends upon whether the transfer can be buffered as indicated when **HPROT[2]** is HIGH for the transfer. Non-buffered writes take a minimum of 3 clock cycles, consisting of the data and at least two inserted wait states per unit transfer. Buffered writes complete in a single cycle when the pipeline is filled. If necessary, the slave can extend the transfer using the **HREADYM** signal.

———— **Note** ————

SINGLE transfers and the initial transfer(s) of a burst take longer than the number of cycles quoted here, because of pipeline fill-latency and bus arbitration.

3.12.6 Bridge restrictions

The following restrictions apply when using the bridge:

- The local arbiter on bus 1 must not break up a fixed-length burst transfer in progress across the bridge, because there is no provision to cancel it on bus 2. The master can either hold off or complete. If the arbiter cannot meet this restriction, or if a master on bus 1 is allowed to terminate the burst in response to an ERROR, **HBURSTM[2:0]** must be externally set to INCR.
- It is an AHB recommendation that slaves do not generate more than 16 wait states, because this might constitute a break-up of the AHB protocol. The slaves on bus 2, accessed by the bridge, must generally adhere to this rule, to avoid excessive hold-up on bus 1. Consideration must be allowed for SPLIT and RETRY responses, when used, because the bridge translates these into wait states on bus 1.
- When a master on bus 1 starts a fixed-length burst, it must not change the state of **HLOCK** mid-way through the transfer due to a restriction of AHB-Lite. (The bridge slave interface receives **HMASTLOCKS** from the arbiter, which is derived from **HLOCK**).

3.13 AHB-Lite to AHB wrapper

The AHB-Lite to AHB wrapper is used to connect an AHB-Lite master to an AHB system. AHB-Lite is a subset of the full AHB specification, intended for use in designs where only a single bus master is used. This can either be:

- a simple single master system
- a multi-layer AHB system where there is only one AHB master per layer.

Even if a master is intended to be used in a full AHB system, it can be designed with an AHB-Lite interface. This reduces the complexity and design time in generating the bus interface. Figure 3-18 shows a block diagram of the AHB-Lite to AHB wrapper.

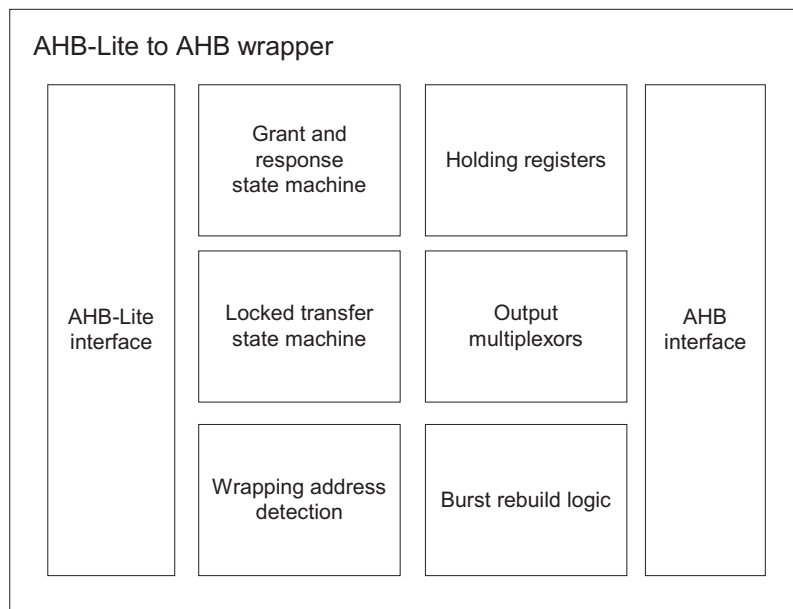


Figure 3-18 AHB-Lite to AHB wrapper block diagram

3.13.1 Signal descriptions

Table 3-11 contains a list of the signals used by the AHB-Lite to AHB wrapper.

Table 3-11 Signal descriptions for AHB-Lite to AHB wrapper

Signal	Type	Direction	Description
MRDATA[31:0]	Read data bus	Input	AHB read data passed through the wrapper to the AHB-Lite interface. Current data corresponds to the previous address on MADDR .
MREADY	Transfer done	Input	When asserted by the AHB wrapper, indicates the current transfer has completed. When deasserted, the wrapper forces the transfer to be extended.
MERROR	Error response	Input	When asserted, indicates an error response from the addressed AHB slave.
MMASTLOCK	Locked sequence	Output	When HIGH indicates that a locked transfer is required. This signal must be asserted a cycle before MTRANS changes from IDLE to NONSEQUENTIAL.
MADDR[31:0]	Address bus	Output	Indicates the required address for the transfer. MADDR is subject to the same restrictions as HADDR (that is, address alignment).
MTRANS[1:0]	Transfer type	Output	Indicates the state of the current transfer. MTRANS assumes the same states as AHB signal HTRANS .
MWRITE	Transfer direction	Output	Indicates a write operation when high and a read operation when LOW. MWRITE has the same function as HWRITE .
MSIZE[2:0]	Transfer size	Output	Indicates the active data lanes for the current transfer. MSIZE assumes the same states as AHB signal HSIZE .
MBURST[2:0]	Burst type	Output	Indicates the type of burst for the current transfer. MBURST assumes the states as AHB signal HBURST .
MPROT[3:0]	Protection control	Output	Protection control signals for the current transfer. MPROT assumes the states as AHB signal HPROT .
MWDATA[31:0]	Write data bus	Output	Data from the file reader, the current data corresponds to the previous address on MADDR .

Note

For a description of the AMBA signals used by the AHB-Lite to AHB wrapper, see *AMBA signals* on page 1-4.

3.14 Interrupt controller (*Interrupt*)

The interrupt controller provides a software interface to the interrupt system. In an ARM system, two levels of interrupt are available:

- *Fast Interrupt Request (FIQ)* for fast, low latency interrupt handling
- *Interrupt Request (IRQ)* for more general interrupts.

Only a single FIQ source at a time is generally used in a system, to provide a true low-latency interrupt. This has the following benefits:

- You can execute the interrupt service routine directly without determining the source of the interrupt.
- Interrupt latency is reduced. You can use the banked registers available for FIQ interrupts more efficiently, because a context save is not required.

There are 32 interrupt lines. The interrupt controller uses a bit position for each different interrupt source. The software can control each request line to generate software interrupts.

Figure 3-19 shows a block diagram of the interrupt controller.

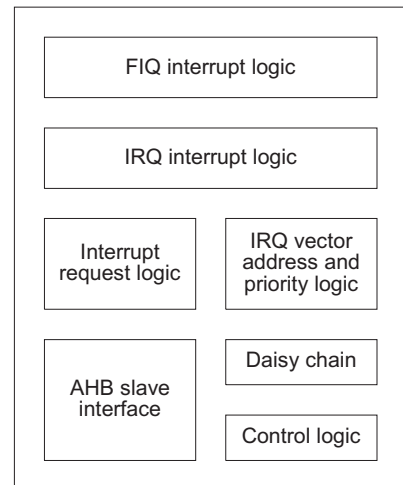


Figure 3-19 Interrupt controller block diagram

The nonvectored and daisy-chained IRQ interrupts provide an address for an *Interrupt Service Routine* (ISR). Reading from the vector interrupt address register, ICVectAddr, provides the address of the ISR, and updates the interrupt priority hardware that masks out the current and any lower priority interrupt requests. Writing to the ICVectAddr register indicates to the interrupt priority hardware that the current interrupt is serviced, allowing lower priority interrupts to go active.

The FIQ interrupt has the highest priority, followed by nonvectored IRQ interrupts. Daisy-chained interrupts have the lowest priority. A programmed interrupt request allows you to generate an interrupt under software control. This register is typically used to downgrade an FIQ interrupt to an IRQ interrupt.

Note

The priority of the FIQ over IRQ is set by the ARM. The interrupt controller can raise both an FIQ and an IRQ at the same time.

The IRQ and FIQ request logic has an asynchronous path. This allows interrupts to be asserted when the clock is disabled.

3.14.1 Programmer's model

By convention, for the IRQ interrupt, bits 1 to 5 must be used as defined in Table 3-12. Bit 0 and bit 6 upwards are available for use as required. For the FIQ interrupt, the bits can be used as required.

Table 3-12 Interrupt standard configuration

Bit	Interrupt source
1	Software interrupt
2	Comms Rx
3	Comms Tx
4	Timer 1
5	Timer 2

The software can control the source interrupt lines to generate software interrupts. These interrupts are generated before interrupt masking, in the same way as external source interrupts. Software interrupts are cleared by writing to the software interrupt clear register, ICSoftIntClear (see *Software interrupt clear register, ICSoftIntClear* on page 3-44). This is normally done at the end of the interrupt service routine.

Interrupt flow sequence

The following procedure shows the sequence for the vectored interrupt flow:

1. An interrupt occurs.
2. The ARM processor branches to either the IRQ or FIQ interrupt vector.
3. If the interrupt is an IRQ, read the ICVectAddr register and branch to the interrupt service routine. This can be done using an LDR PC instruction. Reading the ICVectorAddr register updates the interrupt controllers hardware priority register.
4. Stack the workspace so that IRQ interrupts can be re-enabled.
5. Enable the IRQ interrupts so that a higher priority can be serviced.
6. Execute the *Interrupt Service Routine* (ISR).
7. Clear the requesting interrupt in the peripheral, or write to the ICSoftIntClear register if the request was generated by a software interrupt.
8. Disable the interrupts and restore the workspace.
9. Write to the ICVectAddr register. This clears the respective interrupt in the internal interrupt priority hardware.
10. Return from the interrupt. This re-enables the interrupts.

Simple interrupt flow

The following procedure shows how you can use the interrupt controller without using vectored interrupts or the interrupt priority hardware. For example, you can use it for debugging.

1. An interrupt occurs.
2. Branch to IRQ or FIQ interrupt vector.
3. Branch to the interrupt handler.
4. Interrogate the ICIRQStatus register to determine which source generated the interrupt, and prioritize the interrupts if there are multiple active interrupt sources. This takes a number of instructions to compute.
5. Branch to the correct ISR.
6. Execute the ISR.
7. Clear the interrupt. If the request was generated by a software interrupt, the ICSoftIntClear register must be written to.
8. Check the ICIRQStatus register to ensure that no other interrupt is active. If there is an active request go to Step 4.
9. Return from the interrupt.

————— Note —————

If the above flow is used, you must not read or write to the ICVectorAddr register.

To ensure that the vector address register (see *Vector address register, ICVectAddr* on page 3-45) can be read in a single instruction, the IC base address must be 0xFFFFF000, the upper 4K of memory. Placing the IC anywhere else in memory increases interrupt latency as the ARM processor is unable to access the ICVectorAddr register using a single instruction. The offset of any particular register from the base address is fixed.

Table 3-13 shows the interrupt controller memory map.

Table 3-13 Interrupt controller memory map

Address	Type	Width	Reset value	Name	Description
IC base +0x000	Read	32	0x00000000	ICIRQStatus	IRQ status register
IC base +0x004	Read	32	0x00000000	ICFIQStatus	FIQ status register
IC base +0x008	Read	32	-	ICRawIntr	Raw interrupt status register
IC base +0x00C	Read/write	32	0x00000000	ICIntSelect	Interrupt select register
IC base +0x010	Read/write	32	0x00000000	ICIntEnable	Interrupt enable register
IC base +0x014	Write	32	-	ICIntEnClear	Interrupt enable clear register
IC base +0x018	Read/write	32	0x00000000	ICSoftInt	Software interrupt register
IC base +0x01C	Write	32	-	ICSoftIntClear	Software interrupt clear register
IC base +0x020	Read/write	1	0x0	ICProtection	Protection enable register
IC base +0x030	Read/write	32	0x00000000	ICVectAddr	Vector address register
IC base +0x034	Read/write	32	0x00000000	ICDefVectAddr	Default vector address register
IC base +0x300	Read/write	1	-	ICITCR	Test control register
IC base +0x304	Read	2	-	ICITIP1	Test input register (nICIRQIN/nICFIQIN)
IC base +0x308	Read	32	-	ICITIP2	Test input register (ICVECTADDRIN)
IC base +0x30C	Read	2	0x0	ICITOP1	Test output register (nICIRQ/nICFIQ)
IC base +0x310	Read	32	0x00000000	ICITOP2	Test output register (ICVECTADDRROUT)
IC base +0xFE0	Read	8	0x08	ICPeriphID0	Peripheral ID register bits 7:0
IC base +0xFE4	Read	8	0x18	ICPeriphID1	Peripheral ID register bits 15:8
IC base +0xFE8	Read	8	0x04	ICPeriphID2	Peripheral ID register bits 23:16
IC base +0xFEC	Read	8	0x00	ICPeriphID3	Peripheral ID register bits 31:24
IC base +0xFF0	Read	8	0x0D	ICPCellIID0	PrimeCell ID register bits 7:0
IC base +0xFF4	Read	8	0xF0	ICPCellIID1	PrimeCell ID register bits 15:8
IC base +0xFF8	Read	8	0x05	ICPCellIID2	PrimeCell ID register bits 23:16
IC base +0xFFC	Read	8	0xB1	ICPCellIID3	PrimeCell ID register bits 31:24

The following registers are described in this section:

- *IRQ status register, ICIRQStatus* on page 3-41
- *FIQ status register, ICFIQStatus* on page 3-42
- *Raw interrupt status register, ICRawIntr* on page 3-42
- *Interrupt select register, ICIntSelect* on page 3-42
- *Interrupt enable register, ICIntEnable* on page 3-43
- *Interrupt enable clear register, ICIntEnClear* on page 3-43
- *Software interrupt register, ICSoftInt* on page 3-43
- *Software interrupt clear register, ICSoftIntClear* on page 3-44
- *Protection enable register, ICProtection* on page 3-44
- *Vector address register, ICVectAddr* on page 3-45
- *Default vector address register, ICDefVectAddr* on page 3-45
- *Test control register, ICITCR* on page 3-46
- *Test input register (nICIRQIN/nICFIQIN), ICITIP1* on page 3-46
- *Test input register (ICVECTADDRIN), ICITIP2* on page 3-46
- *Test output register (nICIRQ/nICFIQ), ICITOP1* on page 3-47
- *Test output register (ICVECTADDRROUT), ICITOP2* on page 3-47
- *Peripheral identification registers, ICPPeriphID0-3* on page 3-48
- *PrimeCell identification registers, ICPCellID0-3* on page 3-50.

IRQ status register, ICIRQStatus

The ICIRQStatus register provides the status of interrupts [31:0] after IRQ masking. Table 3-14 shows the bit assignment of the ICIRQStatus register.

Table 3-14 ICIRQStatus register

Bits	Name	Type	Function
31:0	IRQStatus	Read	Shows the status of the interrupts after masking by the ICIntEnable and ICIntSelect registers. A HIGH bit indicates that the interrupt is active, and generates an interrupt to the processor.

FIQ status register, ICFIQStatus

The ICFIQStatus register provides the status of the interrupts after FIQ masking. Table 3-15 shows the bit assignment of the ICFIQStatus register.

Table 3-15 ICFIQStatus register

Bits	Name	Type	Function
31:0	FIQStatus	Read	Shows the status of the interrupts after masking by the ICIntEnable and ICIntSelect registers. A HIGH bit indicates that the interrupt is active, and generates an interrupt to the processor.

Raw interrupt status register, ICRawIntr

The ICRawIntr register provides the status of the source interrupts (and software interrupts) to the interrupt controller. Table 3-16 shows the bit assignment of the ICRawIntr register.

Table 3-16 ICRawIntr register

Bits	Name	Type	Function
31:0	RawInterrupt	Read	Shows the status of the interrupts before masking by the enable registers. A HIGH bit indicates that the appropriate interrupt request is active before masking.

Interrupt select register, ICIntSelect

The ICIntSelect register selects whether the corresponding interrupt source generates an FIQ or an IRQ interrupt. Table 3-17 shows the bit assignment of the ICIntSelect register.

Table 3-17 ICIntSelect register

Bits	Name	Type	Function
31:0	IntSelect	Read/write	Selects type of interrupt for interrupt request: 1 = FIQ interrupt 0 = IRQ interrupt.

Interrupt enable register, ICIntEnable

The ICIntEnable register enables the interrupt request lines, by masking the interrupt sources for the IRQ interrupt. Table 3-18 shows the bit assignment of the ICIntEnable register.

Table 3-18 ICIntEnable register

Bits	Name	Type	Function
31:0	IntEnable	Read/write	Enables the interrupt request lines: 1 = Interrupt enabled. Allows interrupt request to processor. 0 = Interrupt disabled. On reset, all interrupts are disabled. A HIGH bit sets the corresponding bit in the ICIntEnable register. A LOW bit has no effect.

Interrupt enable clear register, ICIntEnClear

The ICIntEnClear register clears bits in the ICIntEnable register. Table 3-19 shows the bit assignment of the ICIntEnClear register.

Table 3-19 ICIntEnClear register

Bits	Name	Type	Function
31:0	IntEnable Clear	Write	Clears bits in the ICIntEnable register. A HIGH bit clears the corresponding bit in the ICIntEnable register. A LOW bit has no effect.

Software interrupt register, ICSoftInt

The ICSoftInt register is used to generate software interrupts. Table 3-20 shows the bit assignment of the ICSoftInt register.

Table 3-20 ICSoftInt register

Bits	Name	Type	Function
31:0	SoftInt	Read/write	Setting a bit generates a software interrupt for the specific source interrupt before interrupt masking. A HIGH bit sets the corresponding bit in the ICSoftInt register. A LOW bit has no effect.

Software interrupt clear register, ICSoftIntClear

The ICSoftIntClear register clears bits in the ICSoftInt register. Table 3-21 shows the bit assignment of the ICSoftIntClear register.

Table 3-21 ICSoftIntClear register

Bits	Name	Type	Function
31:0	SoftIntClear	Write	Clears bits in the ICSoftInt register. A HIGH bit clears the corresponding bit in the ICSoftInt register. A LOW bit has no effect.

Protection enable register, ICProtection

The ICProtection register enables or disables protected register access. Table 3-22 shows the bit assignment of the ICProtection register.

Table 3-22 ICProtection register

Bits	Name	Type	Function
31:1	Reserved	-	-
0	Protection	Read/write	Enables or disables protected register access. When enabled, only privileged mode accesses (reads and writes) can access the interrupt controller registers. When disabled, both User mode and privileged mode can access the registers. This register is cleared on reset, and can only be accessed in privileged mode.

————— Note —————

If the bus master cannot generate accurate protection information, leave this register in its reset state to allow User mode access.

Vector address register, ICVectAddr

The ICVectAddr register contains the *Interrupt Service Routine* (ISR) address of the currently active interrupt. Table 3-23 shows the bit assignment of the ICVectAddr register.

Table 3-23 ICVectAddr register

Bits	Name	Type	Function
31:0	VectorAddr	Read/write	Contains the address of the currently active ISR. Any writes to this register clear the interrupt.

Note

Reading from this register provides the address of the ISR, and indicates to the priority hardware that the interrupt is being serviced. Writing to this register indicates to the priority hardware that the interrupt has been serviced. The register should be used as follows:

- The ISR reads the ICVectAddr register when an IRQ interrupt is generated
- At the end of the ISR, the ICVectAddr register is written to, to update the priority hardware.

Reading or writing to the register at other times can cause incorrect operation.

Default vector address register, ICDefVectAddr

The ICDefVectAddr register contains the default ISR address. Table 3-24 shows the bit assignment of the ICDefVectAddr register.

Table 3-24 ICDefVectAddr register

Bits	Name	Type	Function
31:0	Default VectorAddr	Read/write	Contains the address of the default ISR handler.

Test control register, ICITCR

The ICITCR register selects test mode, and is cleared on reset. Table 3-25 shows the bit assignment of the ICITCR register.

Table 3-25 ICITCR register

Bits	Name	Type	Function
31:1	Reserved	-	-
0	ITEN	Read/write	Selects test mode, to use ICITIP test registers in place of input signals.

Test input register (nICIRQIN/nICFIQIN), ICITIP1

The ICITIP1 register indicates the status of the **nICIRQIN** and **nICFIQIN** daisy chain input lines. Table 3-26 shows the bit assignment of the ICITIP1 register.

Table 3-26 ICITIP1 register

Bits	Name	Type	Function
31:8	Reserved	-	-
7	I	Read	Indicates status of nICIRQIN when ICITCR register is LOW
6	F	Read	Indicates status of nICFIQIN when ICITCR register is LOW
5:0	Reserved	-	-

Test input register (ICVECTADDRIN), ICITIP2

The ICITIP2 register indicates the status of the **ICVECTADDRIN** daisy chain input lines. Table 3-27 shows the bit assignment of the ICITIP2 register.

Table 3-27 ICITIP2 register

Bits	Name	Type	Function
31:0	VectorAddrIn	Read	Indicates status of ICVECTADDRIN when ICITCR register is LOW.

Test output register (nICIRQ/nICFIQ), ICITOP1

The ICITOP1 register indicates the status of the **nICIRQ** and **nICFIQ** interrupt request lines to the processor. Table 3-28 shows the bit assignment of the ICITOP1 register.

Table 3-28 ICITOP1 register

Bits	Name	Type	Function
31:8	Reserved	-	-
7	I	Read	Status of nICIRQ interrupt line. If set HIGH, the interrupt request is active.
6	F	Read	Status of nICFIQ interrupt line. If set HIGH, the interrupt request is active.
5:0	Reserved	-	-

Test output register (ICVECTADDRROUT), ICITOP2

The ICITOP2 register indicates the status of the **ICVECTADDRROUT** lines from the interrupt controller. Table 3-29 shows the bit assignment of the ICITOP2 register.

Table 3-29 ICITOP2 register

Bits	Name	Type	Function
31:0	VectorAddr Out	Read	Indicates status of ICVECTADDRROUT from interrupt controller.

Peripheral identification registers, ICPeriphID0-3

The ICPeriphID0-3 registers are four 8-bit registers, that span address locations 0xFE0-0xFEC. The registers can conceptually be treated as a single 32-bit register. The read-only registers provide the following options of the peripheral:

Part number [11:0]

This identifies the peripheral. The three digit product code 0x90 is used for the interrupt controller.

Designer [19:12]

This is the identification of the designer. ARM Ltd is 0x41 (ASCII A).

Revision number [23:20]

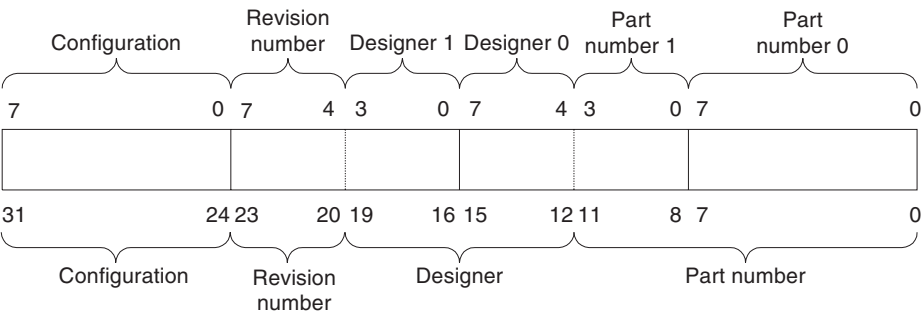
This is the revision number of the peripheral. The revision number starts from 0.

Configuration [31:24]

This is the configuration option of the peripheral. The configuration value is 0.

Figure 3-20 shows the bit assignment for the ICPeriphID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 3-20 Peripheral identification register bit allocation

The four 8-bit peripheral identification registers are described in the following sections:

- *ICPeriphID0 register* on page 3-49
- *ICPeriphID1 register* on page 3-49
- *ICPeriphID2 register* on page 3-50
- *ICPeriphID3 register* on page 3-50.

ICPeriphID0 register

The ICPeriphID0 register is hard-coded and the fields within the register determine the reset value. Table 3-30 shows the bit assignment of the ICPeriphID0 register.

Table 3-30 ICPeriphID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	Partnumber0	Read	These bits read back as 0x90.

ICPeriphID1 register

The ICPeriphID1 register is hard-coded and the fields within the register determine the reset value. Table 3-31 shows the bit assignment of the ICPeriphID1 register.

Table 3-31 ICPeriphID1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:4	Designer0	Read	These bits read back as 0x1.
3:0	Partnumber1	Read	These bits read back as 0x1.

ICPeriphID2 register

The ICPeriphID2 register is hard-coded and the fields within the register determine the reset value. Table 3-32 shows the bit assignment of the ICPeriphID2 register.

Table 3-32 ICPeriphID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:4	Revision	Read	These bits read back as 0x0.
3:0	Designer1	Read	These bits read back as 0x4.

ICPeriphID3 register

The ICPeriphID3 register is hard-coded and the fields within the register determine the reset value. Table 3-33 shows the bit assignment of the ICPeriphID3 register.

Table 3-33 ICPeriphID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	Configuration	Read	These bits read back as 0x0.

PrimeCell identification registers, ICPCellID0-3

The ICPCellID0-3 registers are four 8-bit registers, that span address locations 0xFF0-0xFFC. The read-only register can conceptually be treated as a single 32-bit register. The register is used as a standard cross-peripheral identification system. Figure 3-21 shows the bit assignment for the ICPCellID0-3 registers.

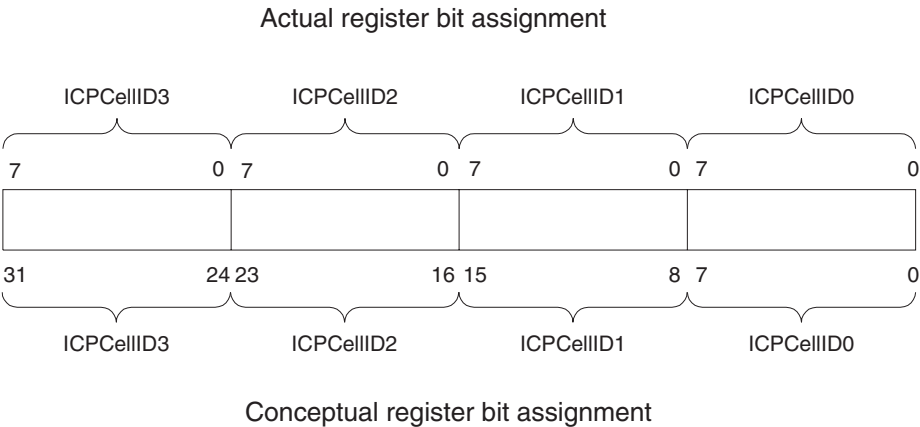


Figure 3-21 PrimeCell identification register bit assignment

The four 8-bit registers are described in the following subsections:

- *ICPCellIID0 register* on page 3-51
- *ICPCellIID1 register* on page 3-52
- *ICPCellIID2 register* on page 3-52
- *ICPCellIID3 register* on page 3-52.

ICPCellIID0 register

The ICPCellIID0 register is hard-coded and the fields within the register determine the reset value. Table 3-34 shows the bit assignment of the ICPCellIID0 register.

Table 3-34 ICPCellIID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	ICPCellIID0	Read	These bits read back as 0x0D.

ICPCellIID1 register

The ICPCellIID1 register is hard-coded and the fields within the register determine the reset value. Table 3-35 shows the bit assignment of the ICPCellIID1 register.

Table 3-35 ICPCellIID1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	ICPCellIID1	Read	These bits read back as 0xF0.

ICPCellIID2 register

The ICPCellIID2 register is hard-coded and the fields within the register determine the reset value. Table 3-36 shows the bit assignment of the ICPCellIID2 register.

Table 3-36 ICPCellIID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	ICPCellIID2	Read	These bits read back as 0x05.

ICPCellIID3 register

The ICPCellIID3 register is hard-coded and the fields within the register determine the reset value. Table 3-37 shows the bit assignment of the ICPCellIID3 register.

Table 3-37 ICPCellIID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must be written as zeros.
7:0	ICPCellIID3	Read	These bits read back as 0xB1.

3.14.2 Signal descriptions

Table 3-38 contains a list of non-AMBA signals used by the interrupt controller.

Table 3-38 Signal descriptions for interrupt controller

Signal	Type	Direction	Description
HSELIC	Input	Decoder	Slave select signal. This is a combinatorial decode of the address bus. It indicates that the current transfer is intended for the selected slave.
ICINTSOURCE [31:0]	Input	Peripheral interrupt request	Interrupt source input
nICIRQ	Output	Interrupt controller	Interrupt request to processor
nICFIQ	Output	Interrupt controller	Fast interrupt request to processor
ICVECTADDRIN [31:0]	Input	External interrupt controller	Connects to the ICVECTADDRROUT [31:0] signal of the previous interrupt controller if daisy chaining is used. Connects to logic 0 if the interrupt controller is not daisy-chained.
ICVECTADDRROUT [31:0]	Output	Interrupt controller	Connects to the ICVECTADDRIN [31:0] signal of the next interrupt controller if daisy chaining is used. Left unconnected if the interrupt controller is not daisy-chained.
nICIRQIN	Input	External interrupt controller	Connects to the nICIRQ signal of the previous interrupt controller if daisy chaining is used. Connects to logic 1 if the interrupt controller is the last in the daisy chain, or if interrupt controller is not daisy-chained.
nICFIQIN	Input	External interrupt controller	Connects to the nICFIQ signal of the previous interrupt controller if daisy chaining is used. Connects to logic 1 if the interrupt controller is the last in the daisy chain, or if interrupt controller is not daisy-chained.
SCANENABLE	Input	Scan controller	Scan enable
SCANINHCLK	Input	Scan controller	Scan data input for HCLK domain
SCANOUTHCLK	Output	Scan controller	Scan data output for HCLK domain

Note

For a description of the AMBA signals used by the interrupt controller, see *AMBA signals* on page 1-4.

Chapter 4

APB Components

This chapter describes the APB components used in the *AMBA Design Kit* (ADK). It contains the following sections:

- *Remap and pause controller (RemapPause)* on page 4-2
- *Example APB slave* on page 4-10
- *Peripheral to bridge multiplexor (MuxP2B)* on page 4-13
- *Watchdog unit (Watchdog)* on page 4-14
- *Dual input timer (Timers)* on page 4-24.

4.1 Remap and pause controller (*RemapPause*)

The remap and pause controller is an APB slave, providing control of the system boot behavior and low-power wait for interrupt mode. A basic block diagram of the remap and pause controller module is shown in Figure 4-1.

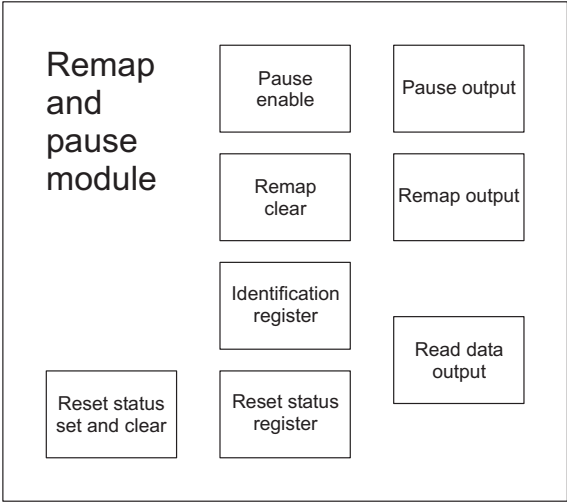


Figure 4-1 Remap and pause module block diagram

4.1.1 Programmer’s model

The base address of the remap and pause controller memory is not fixed and can be different for any particular system implementation. However, the offset of any particular register from the base address is fixed. Table 4-1 shows the remap and pause controller memory map.

Table 4-1 Remap and pause controller memory map

Address	Type	Width	Reset value	Name	Description
RemPause base + 0x00	Write	-	-	Pause	Set PAUSE output
RemPause base + 0x04	Read/write	1	0x0	Remap	Reset memory map (REMAP) output
RemPause base + 0x08	Read/write	8	0x01	ResetStatus	Read: reset status. Write: reset status set.
RemPause base + 0x0C	Write	8	-	ResetStatusClr	Clear bits in reset status.

Table 4-1 Remap and pause controller memory map (continued)

Address	Type	Width	Reset value	Name	Description
RemPause base + 0xFE0	Read	8	0x??	RpcPeriphID0	Peripheral ID register bits 7:0
RemPause base + 0xFE4	Read	8	0x10	RpcPeriphID1	Peripheral ID register bits 15:8
RemPause base + 0xFE8	Read	8	0x04	RpcPeriphID2	Peripheral ID register bits 23:16
RemPause base + 0xFEC	Read	8	0x00	RpcPeriphID3	Peripheral ID register bits 31:24
RemPause base + 0xFF0	Read	8	0x0D	RpcPCellID0	PrimeCell ID register bits 7:0
RemPause base + 0xFF4	Read	8	0xF0	RpcPCellID1	PrimeCell ID register bits 15:8
RemPause base + 0xFF8	Read	8	0x05	RpcPCellID2	PrimeCell ID register bits 23:16
RemPause base + 0xFFC	Read	8	0xB1	RpcPCellID3	PrimeCell ID register bits 31:24

The following registers are described in this section:

- *Pause register, Pause* on page 4-3
- *Remap register, Remap* on page 4-3
- *Reset status register, ResetStatus* on page 4-3
- *Reset status clear register, ResetStatusClr* on page 4-4
- *Peripheral identification registers, RpcPeriphID0-3* on page 4-4
- *PrimeCell identification registers, RpcPCellID0-3* on page 4-7.

Pause register, Pause

Any write to this location sets the **PAUSE** output high. The exact effect of writing to this location is not defined, but typically this would prevent the processor from fetching further instructions until the receipt of an interrupt of a power-on reset.

Remap register, Remap

Bit 0 of this register drives the **REMAP** output. This is typically used to change the memory map from that required during boot-up to that for normal operation. This bit is cleared at reset and once set, can only be cleared again by resetting the block.

Reset status register, ResetStatus

This register provides the reset status of the system. Only bit 0 is defined in this specification, which provides the **PRESETn** status:

- set HIGH at reset
- set LOW through the ResetStatusClr register.

Further bits in the ResetStatus register can be implemented to provide more detailed reset information. The ResetStatus register has a dual mechanism for setting and clearing bits, enabling independent bits to be altered with no knowledge of the other bits in the register. A write to the ResetStatus register has the effect of setting all the bits (except bit 0) which have a corresponding HIGH in the write data.

Reset status clear register, ResetStatusClr

This register location is used to clear bits in the ResetStatus register. Each HIGH bit in the write data to this location, causes the corresponding ResetStatus bit to be cleared.

Peripheral identification registers, RpcPeriphID0-3

The RpcPeriphID0-3 registers are four 8-bit registers, that span address locations 0xFE0 to 0xFEC. The registers can conceptually be treated as a single 32-bit register. The read-only registers provide the following options for the peripheral:

Part number [11:0] This is used to identify the peripheral. The three digit product code 0x809 is used for the remap and pause controller.

Designer [19:12] This is the identification of the designer. ARM Limited is 0x41 (ASCII A).

Revision number [23:20]

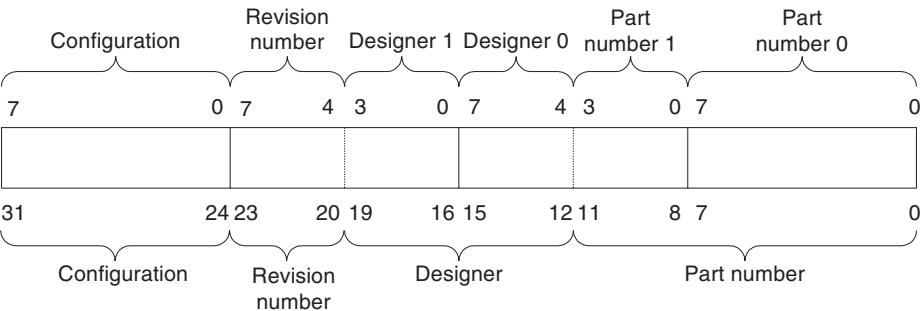
This is the revision number of the peripheral. The revision number starts from 0.

Configuration [31:24]

This is the configuration option of the peripheral. The configuration value is 0.

Figure 4-7 shows the bit assignment for the RpcPeriphID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 4-2 Peripheral identification register bit assignment

————— **Note** —————

When you design a systems memory map you must remember that the register has a 4KB-memory footprint. All memory accesses to the peripheral identification registers must be 32-bit, using the LDR and STR instructions.

The four 8-bit peripheral identification registers are described in the following subsections:

- *RpcPeriphID0* register on page 4-5
- *RpcPeriphID1* register on page 4-6
- *RpcPeriphID2* register on page 4-6
- *RpcPeriphID3* register on page 4-6.

RpcPeriphID0 register

The RpcPeriphID0 register is hard-coded and the fields within the register determine the reset value. Table 4-2 shows the bit assignment of the RpcPeriphID0 register.

Table 4-2 RpcPeriphID0 register

Bits	Name	Description
15:8	-	Reserved, read undefined must read as zeros
7:0	PartNumber0	These bits read back as 0x22

RpcPeriphID1 register

The RpcPeriphID1 register is hard-coded and the fields within the register determine the reset value. Table 4-3 shows the bit assignment of the RpcPeriphID1 register.

Table 4-3 RpcPeriphID1 register

Bits	Name	Description
15:8	-	Reserved, read undefined, must read as zeros
7:4	Designer0	These bits read back as 0x1
3:0	PartNumber1	These bits read back as 0x0.

RpcPeriphID2 register

The RpcPeriphID2 register is hard-coded and the fields within the register determine the reset value. Table 4-4 shows the bit assignment of the RpcPeriphID2 register.

Table 4-4 RpcPeriphID2 register

Bits	Name	Description
15:8	-	Reserved, read undefined, must read as zeros
7:4	Revision	These bits return the peripheral revision
3:0	Designer1	These bits read back as 0x4

RpcPeriphID3 register

The RpcPeriphID3 register is hard-coded and the fields within the register determine the reset value. Table 4-5 shows the bit assignment of the RpcPeriphID3 register.

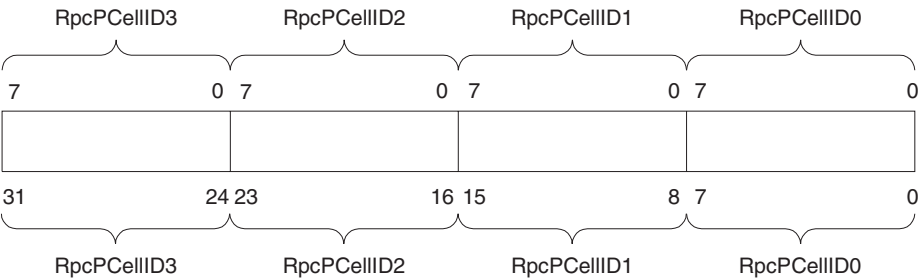
Table 4-5 RpcPeriphID3 register

Bits	Name	Description
15:8	-	Reserved, read undefined, must read as zeros
7:0	Configuration	These bits read back as 0x00

PrimeCell identification registers, RpcPCellID0-3

The RpcPCellID0-3 registers are four 8-bit wide registers, that span address locations 0xFF0-0xFFC. The registers can conceptually be treated as a 32-bit register. The register is used as a standard cross-peripheral identification system. The RpcPCellID register is set to 0xB105F00D. Figure 4-3 shows the bit assignment for the RpcPCellID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 4-3 PrimeCell identification register bit assignment

The four, 8-bit PrimeCell identification registers are described in the following subsections:

- *RpcPCellID0* register on page 4-7
- *RpcPCellID1* register on page 4-8
- *RpcPCellID2* register on page 4-8
- *RpcPCellID3* register on page 4-8.

RpcPCellID0 register

The RpcPCellID0 register is hard-coded and the fields within the register determine the reset value. Table 4-6 shows the bit assignment of the RpcPCellID0 register.

Table 4-6 RpcPCellID0 register

Bits	Name	Description
15:8	-	Reserved, read undefined, must read as zeros
7:0	RpcPCellID0	These bits read back as 0x0D

RpcPCellID1 register

The RpcPCellID1 register is hard-coded and the fields within the register determine the reset value. Table 4-7 shows the bit assignment of the RpcPCellID1 register.

Table 4-7 RpcPCellID1 register

Bits	Name	Description
15:8	-	Reserved, read undefined, must read as zeros
7:0	RpcPCellID1	These bits read back as 0xF0

RpcPCellID2 register

The RpcPCellID2 register is hard-coded and the fields within the register determine the reset value. Table 4-8 shows the bit assignment of the RpcPCellID2 register.

Table 4-8 RpcPCellID2 register

Bits	Name	Description
15:8	-	Reserved, read undefined, must read as zeros
7:0	RpcPCellID2	These bits read back as 0x05

RpcPCellID3 register

The RpcPCellID3 register is hard-coded and the fields within the register determine the reset value. Table 4-9 shows the bit assignment of the RpcPCellID3 register.

Table 4-9 RpcPCellID3 register

Bits	Name	Description
15:8	-	Reserved, read undefined, must read as zeros
7:0	RpcPCellID3	These bits read back as 0xB1

4.1.2 Signal descriptions

Table 4-10 contains a list of non-AMBA signals used by the remap and pause controller.

Table 4-10 Signal descriptions for remap and pause controller

Signal	Type	Direction	Description
nFIQ	FIQ output	Input	FIQ interrupt input from the interrupt controller.
nIRQ	IRQ output	Input	IRQ interrupt input from the interrupt controller.
Pause	Pause mode	Output	HIGH when in the <i>wait for interrupt</i> pause mode, and LOW at all other times.
Remap	Reset memory map	Output	LOW when the reset memory map is in use, and HIGH when the normal memory map is in use.

———— **Note** ————

For a description of the AMBA signals used by the remap and pause controller, see *AMBA signals* on page 1-4.

4.2 Example APB slave

The example APB slave provides sample HDL code, which can be used as a basis for further enhancement, to produce a slave on the APB. The example APB slave consists of five functional blocks contained within one top-level entity, as shown in Figure 4-4.

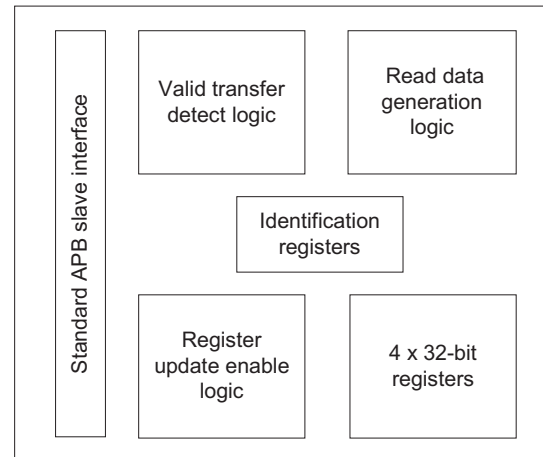


Figure 4-4 Example APB slave block diagram

The example APB slave includes the following features:

- A simple APB slave interface.
- SYSTEM bus clock and reset, which are synchronous between AHB and APB domains.
- 32-bit data bus (endian-independent, but data handling is 32-bits only).
- DATA transfers require two clock cycles.
- 32-bit address bus.
- FOUR 32-bit registers to hold the write data. Registers can also be read.
- SEVEN read-only locations that return logical functions of the registered write data.

4.2.1 Programmer's model

The slave only responds to a transfer when not in reset (**PRESETn** high), and when **PSEL** is HIGH and **PENABLE** is LOW at the time of a rising edge on **PCLK**. The timing of the slave corresponds to the APB specification.

The functionality of the slave depends on the address location being accessed and the direction of the data. Only bits [11:2] of the address are decoded, which supports accesses with word-aligned addresses. Because of the partial address decode, the same location is accessible at several addresses. For example, Register 0 is accessible at 0x0000, 0x1000, 0x2000 etc, until the decode range of **PSEL** from the bridge is exceeded.

Table 4-11 shows the example APB slave memory map

Table 4-11 Example APB slave memory map

Address	Type	Width	Reset value	Name	Description
Base + 0x00	Read/ write	32	0x00000000	R0	Read/write data into R0.
Base + 0x04	Read/ write	32	0x00000000	R1	Read/write data into R1.
Base + 0x08	Read/ write	32	0x00000000	R2	Read/write data into R2.
Base + 0x0C	Read/ write	32	0x00000000	R3	Read/write data into R3.
Base + 0x10	Read	32	0xFFFFFFFF	Read10	Not R0.
Base + 0x14	Read	32	0x00000000	Read14	R0 and R1.
Base + 0x18	Read	32	0x00000000	Read18	R1 or R2.
Base + 0x1C	Read	32	0x00000000	Read1C	R2 xor R3.
Base + 0x20	Read	32	0x00000000	Read20	R0 and R1 and R2 and R3.
Base + 0x24	Read	32	0x00000000	Read24	R0 or R1 or R2 or R3.
Base + 0x28	Read	32	0x00000000	Read28	R0 xor R1 xor R2 xor R3.
Base + 0x2C to 0xFDC	Read/ write	32	0x00000000	Reserved	Read as zero. Write has no effect.
Base + 0xFE0	Read	8	0x06	PeripheralID0	Peripheral ID register 0.
Base + 0xFE4	Read	8	0x18	PeripheralID1	Peripheral ID register 1.
Base + 0xFE8	Read	8	0x04	PeripheralID2	Peripheral ID register 2.

Table 4-11 Example APB slave memory map (continued)

Address	Type	Width	Reset value	Name	Description
Base + 0xFEC	Read	8	0x00	PeripheralID3	Peripheral ID register 3.
Base + 0xFF0	Read	8	0x0D	PrimeCellID0	PrimeCell ID register 0.
Base + 0xFF4	Read	8	0xF0	PrimeCellID1	PrimeCell ID register 1.
Base + 0xFF8	Read	8	0x05	PrimeCellID2	PrimeCell ID register 2.
Base + 0xFFC	Read	8	0xB1	PrimeCellID3	PrimeCell ID register 3.

The functionality in Table 4-11 is the same as the retry slave detailed in *Retry slave* on page 3-7, except there is no requirement to generate wait states and responses, and all data is a fixed size (32-bit).

Note

Address locations 0x2C to 0xFDC read as zero and perform no operation when written to.

The peripheral ID information is arranged as four 8-bit registers that span address locations 0xFE0 to 0xFEC. The registers can conceptually be treated as a 32-bit read-only register. Table 4-12 shows the format of this information and the values for the peripheral.

Table 4-12 Peripheral ID format

Register field	Description
Part number [11:0]	This is used to identify the peripheral. The three digit product code 0x806 is used.
Designer ID [19:12]	This is the identification of the designer. ARM Ltd is 0x41 (ASCII A).
Revision [23:20]	This is the revision number of the peripheral. The revision number starts from 0.
Configuration [31:24]	This is the configuration option of the peripheral. The configuration value is 0.

4.2.2 Signal descriptions

The example APB slave uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

4.3 Peripheral to bridge multiplexor (*MuxP2B*)

The peripheral to bridge multiplexor is used to connect the read data outputs of each APB slave to the APB bridge module, using the **PSELx** signals to select the required data source. The default configuration is sixteen slots. The read data is switched for the duration of an APB transfer, when the **PSELx** signal is valid. A default value of zero is used when no slaves are selected. Figure 4-5 shows the peripheral to bridge multiplexor block diagram.

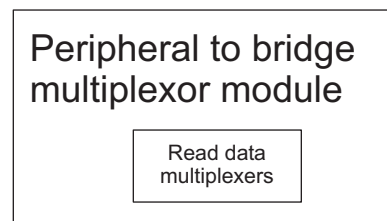


Figure 4-5 Peripheral to bridge multiplexor module block diagram

4.3.1 Signal descriptions

The peripheral to bridge multiplexor uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

4.4 Watchdog unit (*Watchdog*)

The watchdog unit provides a way of recovering from software crashes. The watchdog clock is used to generate a regular interrupt (**WDOGIN**T), depending on a programmed value. The watchdog monitors the interrupt and asserts a reset signal (**WDOGRES**) if the interrupt remains unserviced for the entire programmed period. You can enable or disable the watchdog unit as required. Figure 4-6 shows the watchdog block diagram.

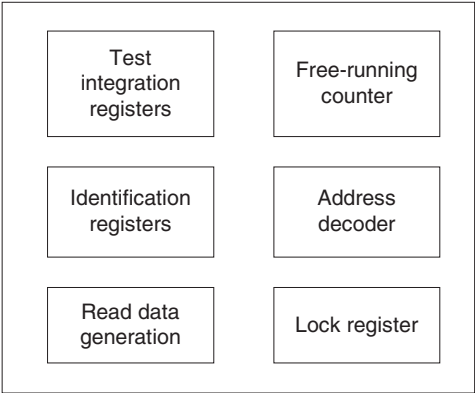


Figure 4-6 Watchdog block diagram

4.4.1 Programmer’s model

The watchdog registers are shown in Table 4-13.

Table 4-13 Watchdog unit register summary

Address	Type	Width	Reset value	Name	Description
Wdog base + 0x00	Read/Write	32	0xFFFFFFFF	WdogLoad	Watchdog load register
Wdog base + 0x04	Read Only	32	0xFFFFFFFF	WdogValue	The current value for the watchdog counter
Wdog base + 0x08	Read/Write	2	0x0	WdogControl	Watchdog control register
Wdog base + 0x0C	Write Only	-	-	WdogIntClr	Clears the watchdog interrupt
Wdog base + 0x10	Read Only	1	0x0	WdogRIS	Watchdog raw interrupt status
Wdog base + 0x14	Read Only	1	0x0	WdogMIS	Watchdog masked interrupt status

Table 4-13 Watchdog unit register summary (continued)

Address	Type	Width	Reset value	Name	Description
Wdog base + 0xC00	Read/Write	32	0x0	WdogLock	Watchdog lock register
Wdog base + 0xF00	Read/Write	1	0x0	WdogITCR	Integration test control register
Wdog base + 0xF04	Write Only	2	0x0	WdogITOP	Integration test output set register
Wdog base + 0xFE0	Read Only	8	0x05	WdogPeriphID0	Peripheral ID register bits 7:0
Wdog base + 0xFE4	Read Only	8	0x18	WdogPeriphID1	Peripheral ID register bits 15:8
Wdog base + 0xFE8	Read Only	8	0x04	WdogPeriphID2	Peripheral ID register bits 23:16
Wdog base + 0xFEC	Read Only	8	0x00	WdogPeriphID3	Peripheral ID register bits 31:24
Wdog base + 0xFF0	Read Only	8	0x0D	WdogPCellID0	PrimeCell ID register bits 7:0
Wdog base + 0xFF4	Read Only	8	0xF0	WdogPCellID1	PrimeCell ID register bits 15:8
Wdog base + 0xFF8	Read Only	8	0x05	WdogPCellID2	PrimeCell ID register bits 23:16
Wdog base + 0xFFC	Read Only	8	0xB1	WdogPCellID3	PrimeCell ID register bits 31:24

The following registers are described in this section:

- *Watchdog load register, WdogLoad* on page 4-15
- *Watchdog control register, WdogControl* on page 4-16
- *Watchdog clear interrupt register, WdogIntClr* on page 4-16
- *Raw interrupt status register, WdogRIS* on page 4-16
- *Interrupt status register, WdogMIS* on page 4-17
- *Watchdog lock register, WdogLock* on page 4-17
- *Integration test control register, WdogITCR* on page 4-18
- *Integration test output set register, WdogITOP* on page 4-18
- *Peripheral identification registers, WdogPeriphID0-3* on page 4-18
- *PrimeCell identification registers, WdogPCellID0-3* on page 4-21.

Watchdog load register, WdogLoad

This is a 32-bit register containing the value from which the counter is to decrement. When this register is written to, the count is immediately restarted from the new value. The minimum valid value for WdogLoad is one.

Watchdog control register, WdogControl

This is a read/write register that enables the software to control the watchdog unit. Table 4-14 shows the bit assignment of the WdogControl register.

Table 4-14 WdogControl register

Bits	Name	Type	Function
1	RESEN	Read/write	Enable Watchdog reset output (WDOGRES). Acts as a mask for the reset output. Set HIGH to enable the reset, and LOW to disable the reset.
0	INTEN	Read/write	Enable the interrupt event (WDOGINT). Set HIGH to enable the counter and the interrupt, and set LOW to disable the counter and interrupt. Reloads the counter from the value in WdogLoad when the interrupt is enabled, and was previously disabled.

Watchdog clear interrupt register, WdogIntClr

A write of any value to this location clears the watchdog interrupt, and reloads the counter from the value in WdogLoad.

Raw interrupt status register, WdogRIS

This register indicates the raw interrupt status from the counter. This value is ANDed with the interrupt enable bit from the control register to create the masked interrupt, which is passed to the interrupt output pin. Table 4-15 shows the bit assignment of the WdogRIS register.

Table 4-15 WdogRIS register

Bits	Name	Type	Function
0	Raw Watchdog Interrupt	Read	Raw interrupt status from the counter

Interrupt status register, WdogMIS

This register indicates the masked interrupt status from the counter. This value is the logical AND of the raw interrupt status with the INTEN bit from the control register, and is the same value which is passed to the interrupt output pin. Table 4-16 shows the bit assignment of the WdogMIS register.

Table 4-16 WdogMIS register

Bits	Name	Type	Function
0	Watchdog Interrupt	Read	Enabled interrupt status from the counter

Watchdog lock register, WdogLock

Use of this register allows write-access to all other registers to be disabled. This is to prevent rogue software from disabling the watchdog functionality. Writing a value of 0x1ACCE551 will enable write access to all other registers; writing any other value will disable write accesses. A read from this register will return only the bottom bit:

- 0 indicates that write access is enabled (not locked)
- 1 indicates that write access is disabled (locked).

Table 4-17 shows the bit assignment of the WdogLock register.

Table 4-17 WdogLock register

Bits	Name	Type	Function
31:0	Enable register writes	Write	Enable write access to all other registers by writing 0x1ACCE551. Disable write access by writing any other value.
0	Register write enable status	Read	0 - write access to all other registers is enabled (default) 1 - write access to all other registers is disabled

Integration test control register, WdogITCR

Single-bit register used to enable integration test mode. When in this mode, the masked interrupt output, **WDOGIN**T, and reset output, **WDOGRES**, are directly controlled by the test output set register. Table 4-18 shows the bit assignment of the WdogITCR register.

Table 4-18 WdogITCR register

Bits	Name	Type	Function
0	Integration Test Mode Enable	Read/write	When set HIGH, places the Watchdog into integration test mode

Integration test output set register, WdogITOP

When in integration test mode, the enabled interrupt output and reset output are driven directly from the values in this register. Table 4-19 shows the bit assignment of the WdogITOP register.

Table 4-19 WdogITOP register

Bits	Name	Type	Function
1	Integration Test WDOGIN T value	Write	Value output on WDOGIN T when in Integration Test Mode
0	Integration Test WDOGRES value	Write	Value output on WDOGRES when in Integration Test Mode

Peripheral identification registers, WdogPeriphID0-3

The WdogPeriphID0-3 registers are four 8-bit registers, that span address locations 0xFE0-0xFEC. The registers can conceptually be treated as a single 32-bit register. The read-only registers provide the following options for the peripheral:

Part number [11:0] This is used to identify the peripheral. The three digit product code 0x805 is used for the watchdog unit.

Designer [19:12] This is the identification of the designer. ARM Limited is 0x41 (ASCII A).

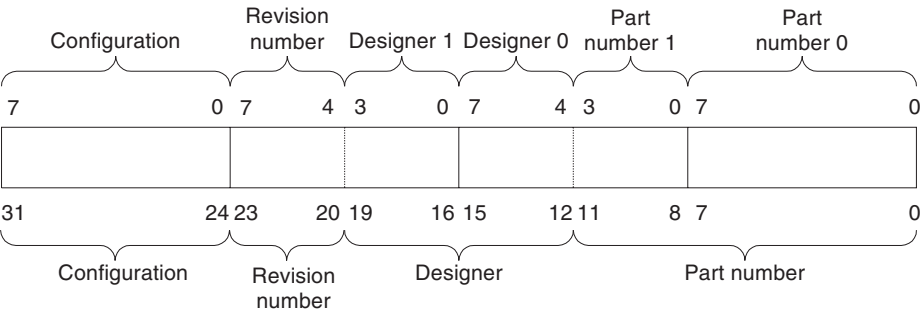
Revision number [23:20]
This is the revision number of the peripheral. The revision number starts from 0.

Configuration [31:24]

This is the configuration option of the peripheral. The configuration value is 0.

Figure 4-7 shows the bit assignment for the WdogPeriphID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 4-7 Peripheral identification register bit assignment

Note

When you design a systems memory map you must remember that the register has a 4KB-memory footprint. All memory accesses to the peripheral identification registers must be 32-bit, using the LDR and STR instructions.

The four 8-bit peripheral identification registers are described in the following subsections:

- *WdogPeriphID0 register* on page 4-20
- *WdogPeriphID1 register* on page 4-20
- *WdogPeriphID2 register* on page 4-20
- *WdogPeriphID3 register* on page 4-21.

WdogPeriphID0 register

The WdogPeriphID0 register is hard-coded and the fields within the register determine the reset value. Table 4-20 shows the bit assignment of the WdogPeriphID0 register.

Table 4-20 WdogPeriphID0 register

Bits	Name	Description
31:8	-	Reserved, read undefined must read as zeros
7:0	PartNumber0	These bits read back as 0x05

WdogPeriphID1 register

The WdogPeriphID1 register is hard-coded and the fields within the register determine the reset value. Table 4-21 shows the bit assignment of the WdogPeriphID1 register.

Table 4-21 WdogPeriphID1 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must read as zeros
7:4	Designer0	These bits read back as 0x1
3:0	PartNumber1	These bits read back as 0x08.

WdogPeriphID2 register

The WdogPeriphID2 register is hard-coded and the fields within the register determine the reset value. Table 4-22 shows the bit assignment of the WdogPeriphID2 register.

Table 4-22 WdogPeriphID2 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must read as zeros
7:4	Revision	These bits read back as 0x0
3:0	Designer1	These bits read back as 0x4

WdogPeriphID3 register

The WdogPeriphID3 register is hard-coded and the fields within the register determine the reset value. Table 4-23 shows the bit assignment of the WdogPeriphID3 register.

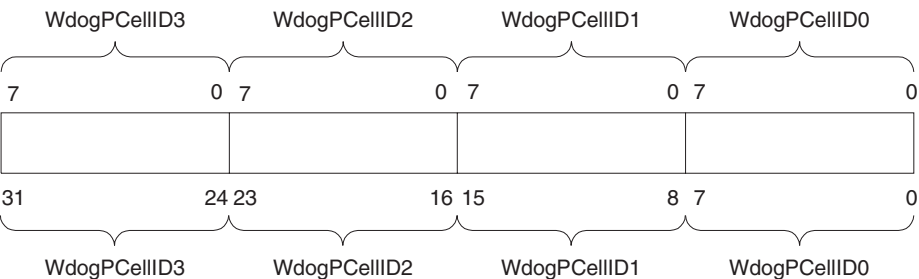
Table 4-23 WdogPeriphID3 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must read as zeros
7:0	Configuration	These bits read back as 0x00

PrimeCell identification registers, WdogPCellID0-3

The WdogPCellID0-3 registers are four 8-bit wide registers, that span address locations 0xFF0-0xFFC. The registers can conceptually be treated as a 32-bit register. The register is used as a standard cross-peripheral identification system. The WdogPCellID register is set to 0xB105F00D. Figure 4-8 shows the bit assignment for the WdogPCellID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 4-8 PrimeCell identification register bit assignment

The four, 8-bit PrimeCell identification registers are described in the following subsections:

- *WdogPCellID0 register* on page 4-22
- *WdogPCellID1 register* on page 4-22
- *WdogPCellID2 register* on page 4-22
- *WdogPCellID3 register* on page 4-23.

WdogPCellID0 register

The WdogPCellID0 register is hard-coded and the fields within the register determine the reset value. Table 4-24 shows the bit assignment of the WdogPCellID0 register.

Table 4-24 WdogPCellID0 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must read as zeros
7:0	WdogPCellID0	These bits read back as 0x0D

WdogPCellID1 register

The WdogPCellID1 register is hard-coded and the fields within the register determine the reset value. Table 4-25 shows the bit assignment of the WdogPCellID1 register.

Table 4-25 WdogPCellID1 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must read as zeros
7:0	WdogPCellID1	These bits read back as 0xF0

WdogPCellID2 register

The WdogPCellID2 register is hard-coded and the fields within the register determine the reset value. Table 4-26 shows the bit assignment of the WdogPCellID2 register.

Table 4-26 WdogPCellID2 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must read as zeros
7:0	WdogPCellID2	These bits read back as 0x05

WdogPCellID3 register

The WdogPCellID3 register is hard-coded and the fields within the register determine the reset value. Table 4-27 shows the bit assignment of the WdogPCellID3 register.

Table 4-27 WdogPCellID3 register

Bits	Name	Description
31:8	-	Reserved, read undefined, must read as zeros
7:0	WdogPCellID3	These bits read back as 0xB1

4.4.2 Signal descriptions

Table 4-28 contains a list of non-AMBA signals used by the watchdog unit.

Table 4-28 Signal descriptions for watchdog unit

Signal	Type	Direction	Description
WDOGCLK	Watchdog clock	Input	The watchdog clock input.
WDOGCLKEN	Watchdog clock enable	Input	The enable for the watchdog clock input. The counters will only decrement on a rising edge of WDOGCLK when WDOGCLKEN is HIGH.
WDOGRESn	Watchdog reset	Input	The watchdog clock domain reset input.
WDOGINT	Watchdog interrupt	Output	The watchdog interrupt.
WDOGRES	Watchdog reset	Output	The watchdog timeout reset.

Note

For a description of the AMBA signals used by the watchdog unit, see *AMBA signals* on page 1-4.

4.5 Dual input timer (*Timers*)

The Dual Input Timers module is an APB slave that provides access to two interrupt-generating, programmable 32-bit *Free-Running decrementing Counters* (FRCs). The system clock (**PCLK**) is used to control the programmable registers, and the second clock input is used to drive the counter, enabling the counters to run from a much slower clock than the system clock. The two clocks must be synchronous while register accesses are performed. A top-level block diagram of the timers is shown in Figure 4-9.

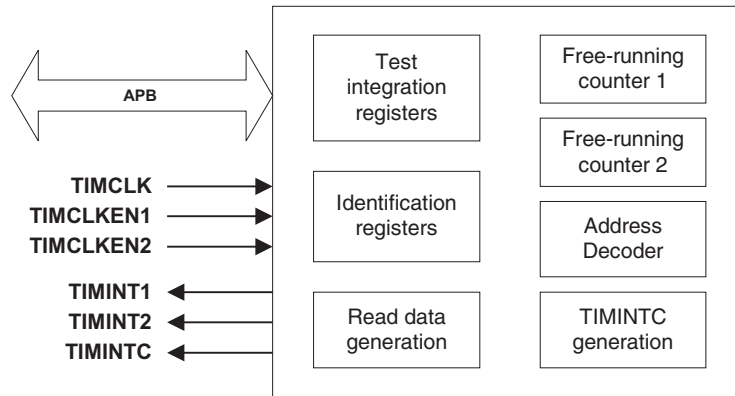


Figure 4-9 Dual input timer block diagram

4.5.1 Functional description

Two timers are defined as the default provided, although this can easily be expanded through extra instantiations of the FRC block. The same principle of simple expansion has been applied to the register configuration, enabling more complex counters to be used. For each timer, the following modes of operation are available:

Free-running mode

The counter wraps after reaching its zero value, and continues to count down from the maximum value. This is the default mode.

Periodic timer mode

The counter generates an interrupt at a constant interval, reloading the original value after wrapping past zero.

One-shot timer mode

The counter generates an interrupt once. When the counter reaches zero, it halts until reprogrammed by the user. This can be achieved by either clearing the One Shot Count bit in the control register (in which case the count will proceed according to the selection of Free-running or Periodic mode), or by writing a new value to the Load Value register.

4.5.2 Operation

Each timer has an identical set of registers as shown in Table 4-29 on page 4-28. The operation of each timer is identical. The timer is loaded by writing to the load register and, if enabled, counts down to zero. When a counter is already running, writing to the load register will cause the counter to immediately restart at the new value. Writing to the background load value has no effect on the current count. The counter continues to decrement to zero, and then recommences from the new load value (if in periodic mode, and one shot mode is not selected).

When zero is reached, an interrupt is generated. The interrupt can be cleared by writing to the clear register. If One Shot Mode is selected, the counter halts on reaching zero until the you deselect One Shot Mode, or write a new Load value. Otherwise, after reaching a zero count, if the timer is operating in free-running mode it continues to decrement from its maximum value. If periodic timer mode is selected, the timer reloads the count value from the load register and continues to decrement. In this mode the counter effectively generates a periodic interrupt. The mode is selected by a bit in the timer control register (see Table 4-30 on page 4-30). At any point, the current counter value can be read from the Value register. The counter is enabled by a bit in the control register. At reset, the counter is disabled, the interrupt is cleared, and the load register is set to zero. The mode and prescale values are set to free-running, and clock divide of 1 respectively. A block diagram of the free-running timer module is shown in Figure 4-10 on page 4-26.

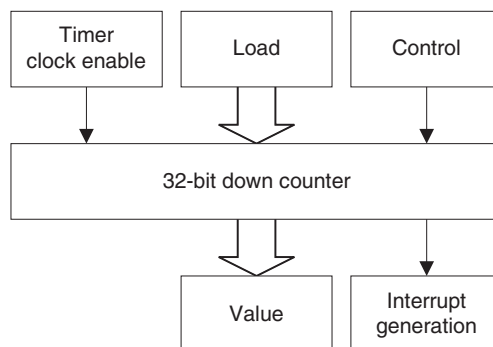


Figure 4-10 Free-running timer block

The timer clock enable is generated by a prescale unit. The enable is then used by the counter to create a clock with a timing of one of the following:

- the system clock
- the system clock divided by 16, generated by 4 bits of prescale
- the system clock divided by 256, generated by a total of 8 bits of prescale.

Figure 4-11 shows how the timer clock frequency is selected in the prescale unit. This enables you to clock the timer at different frequencies.

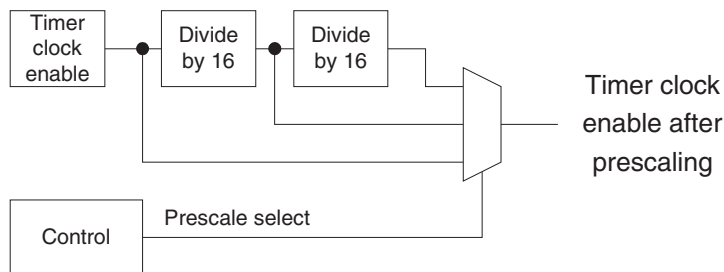


Figure 4-11 Prescale clock enable generation

Note

This selection is in addition to any similar facility already provided as part of any clock generation logic external to the Timers.

Interrupt generation

An interrupt is generated when the full 32-bit counter reaches zero, and is only cleared when the **TimerXClear** location is written to. A register holds the value until the interrupt is cleared. The most significant carry bit of the counter detects the counter reaching zero.

Interrupts can be masked by writing 0 to the Interrupt Enable bit in the Control register. Both the raw interrupt status (prior to masking) and the final interrupt status (after masking) can be read from status registers.

The interrupts from the individual counters (after masking) are logically OR'ed into a combined interrupt, **TIMINTC**, which is provided as an additional output from the Timer peripheral.

4.5.3 Clocking

The timers have two clock inputs, **PCLK** and **TIMCLK**. **PCLK** is the main APB system clock, and is used by the register interface. **TIMCLK** is the input to the prescale units and the decrementing counters. A pulse on **TIMCLK** must be qualified by the appropriate **TIMCLKENx** being HIGH.

The design of the timers assumes that **PCLK** and **TIMCLK** are synchronous. To enable the counter to operate from a lower effective frequency than that at which **PCLK** is running, either of the following can be done:

- both **PCLK** and **TIMCLK** inputs are connected to the APB **PCLK** signal, and **TIMCLKENx** is pulsed HIGH at the required frequency (synchronized to **PCLK**)
- **TIMCLKENx** is tied HIGH and an enabled version of **PCLK** is fed into the **TIMCLK** input, giving sparse clock pulses synchronous to **PCLK**.

This provision of two clock inputs enables the counters to continue to run while the APB system is in a sleep state whereby **PCLK** is disabled. The changeover periods when **PCLK** is disabled and enabled must be handled by external system control logic, to ensure that the **PCLK** and **TIMCLK** inputs are fed with synchronous signals when any register access is to occur.

4.5.4 Programmer's model

The timer registers are shown in Table 4-29.

Table 4-29 Timer register summary

Address	Type	Width	Reset value	Name	Description
Timer base + 0x00	Read/write	32	0x00000000	Timer1Load	Load value for Timer1
Timer base + 0x04	Read	32	0xFFFFFFFF	Timer1Value	The current value for Timer1
Timer base + 0x08	Read/write	8	0x20	Timer1Control	Timer 1 control register
Timer base + 0x0C	Write	-	-	Timer1IntClr	Timer 1 interrupt clear
Timer base + 0x10	Read	1	0x0	Timer1RIS	Timer 1 raw interrupt status
Timer base + 0x14	Read	1	0x0	Timer1MIS	Timer 1 masked interrupt status
Timer base + 0x18	Read/write	32	0x00000000	Timer1BGLoad	Background load value for Timer 1
Timer base + 0x20	Read/write	32	0x00000000	Timer2Load	Load value for Timer 2
Timer base + 0x24	Read	32	0xFFFFFFFF	Timer2Value	The current value for Timer 2
Timer base + 0x28	Read/write	8	0x20	Timer2Control	Timer 2 control register
Timer base + 0x2C	Write	-	-	Timer2IntClr	Timer 2 interrupt clear
Timer base + 0x30	Read	1	0x0	Timer2RIS	Timer 2 raw interrupt status
Timer base + 0x34	Read	1	0x0	Timer2MIS	Timer 2 masked interrupt status
Timer base + 0x38	Read/write	32	0x00000000	Timer2BGLoad	Background load value for Timer 2
Timer base + 0xF00	Read/write	1	0x0	TimerITCR	Integration test control register
Timer base + 0xF04	Write	2	0x0	TimerITOP	Integration test output set register
Timer base + 0xFE0	Read	8	0x04	TimerPeriphID0	Peripheral ID register bits 7:0
Timer base + 0xFE4	Read	8	0x18	TimerPeriphID1	Peripheral ID register bits 15:8
Timer base + 0xFE8	Read	8	0x04	TimerPeriphID2	Peripheral ID register bits 23:16
Timer base + 0xFEC	Read	8	0x00	TimerPeriphID3	Peripheral ID register bits 31:24
Timer base + 0xFF0	Read	8	0x0D	TimerPCellID0	PrimeCell ID register bits 7:0
Timer base + 0xFF4	Read	8	0xF0	TimerPCellID1	PrimeCell ID register bits 15:8
Timer base + 0xFF8	Read	8	0x05	TimerPCellID2	PrimeCell ID register bits 23:16
Timer base + 0xFFC	Read	8	0xB1	TimerPCellID3	PrimeCell ID register bits 31:24

The following registers are described in this section:

- *Load register, TimerXLoad* on page 4-29
- *.Current value register, TimerXValue* on page 4-29
- *Timer control register, TimerXControl* on page 4-30
- *Interrupt clear register, TimerXIntClr* on page 4-30
- *Raw interrupt status register, TimerXRIS* on page 4-31
- *Interrupt status register, TimerXMIS* on page 4-31
- *Background load register, TimerXBGLoad* on page 4-31
- *Integration test control register, TimerITCR* on page 4-32
- *Integration test output set register, TimerITOP* on page 4-32
- *Peripheral identification registers, TimerPeriphID0-3* on page 4-32
- *PrimeCell identification registers, TimerPCellID0-3* on page 4-35.

Load register, TimerXLoad

This is a 32-bit register containing the value from which the counter is to decrement. This is the value used to reload the counter when Periodic mode is enabled, and the current count reaches zero.

When this register is written to directly, the current count is immediately reset to the new value at the next rising edge of **TIMCLK** which is enabled by **TIMCLKEN**.

The value in this register is also overwritten if the TimerXBGLoad register is written to, but the current count is not immediately affected.

If values are written to both the TimerXLoad and TimerXBGLoad registers before an enabled rising edge on **TIMCLK**, the following occurs:

- on the next enabled TIMCLK edge the value written to the TimerXLoad value replaces the current count value
- following this, each time the counter reaches zero, the current count value is reset to the value written to TimerXBGLoad.

Reading from the TimerXLoad register at any time after the two writes have occurred will retrieve the value written to TimerXBGLoad. That is, the value read from TimerXLoad is always the value which will take effect for Periodic mode after the next time the counter reaches zero.

Current value register, TimerXValue

This register gives the current value of the decrementing counter.

Timer control register, TimerXControl

Table 4-30 shows the bit assignments for the TimerXControl register.

Table 4-30 TimerXControl register

Bits	Name	Type	Function
7	Timer Enable	Read/write	Enable bit: 0 = Timer disabled (default) 1 = Timer enabled.
6	Timer Mode	Read/write	Mode bit: 0 = Timer is in free-running mode (default) 1 = Timer is in periodic mode.
5	Interrupt Enable	Read/write	Interrupt Enable bit: 0 = Timer Interrupt disabled 1 = Timer Interrupt enabled (default)
4	RESERVED		Reserved bit, do not modify, and ignore on read
3:2	TimerPre	Read/write	Prescale bits: 00 = 0 stages of prescale, clock is divided by 1 (default) 01 = 4 stages of prescale, clock is divided by 16 10 = 8 stages of prescale, clock is divided by 256 11 = Undefined, do not use.
1	Timer Size	Read/write	Selects 16/32 bit counter operation: 0 = 16-bit counter (default) 1 = 32-bit counter
0	One Shot Count	Read/write	Selects one-shot or wrapping counter mode: 0 = wrapping mode (default) 1 = one-shot mode

Interrupt clear register, TimerXIntClr

Any write to this register will clear the interrupt output from the counter.

Raw interrupt status register, TimerXRIS

This register indicates the raw interrupt status from the counter. This value is ANDed with the timer interrupt enable bit from the control register to create the masked interrupt, which is passed to the interrupt output pin. Table 4-31 shows the bit assignments for the TimerXRIS register.

Table 4-31 TimerXRIS register

Bits	Name	Type	Function
0	Raw Timer Interrupt	Read	Raw interrupt status from the counter

Interrupt status register, TimerXMIS

This register indicates the masked interrupt status from the counter. This value is the logical AND of the raw interrupt status with the timer interrupt enable bit from the control register, and is the same value which is passed to the interrupt output pin. Table 4-32 shows the bit assignments for the TimerXMIS register.

Table 4-32 TimerXMIS register

Bits	Name	Type	Function
0	Timer Interrupt	Read	Enabled interrupt status from the counter

Background load register, TimerXBGLoad

This is a 32-bit register containing the value from which the counter is to decrement. This is the value used to reload the counter when Periodic mode is enabled, and the current count reaches zero.

This register provides an alternative method of accessing the TimerXLoad register. The difference is that writes to TimerXBGLoad will not cause the counter immediately to restart from the new value.

Reading from this register returns the same value returned from TimerXLoad. See *Load register, TimerXLoad* on page 4-29 for more details.

Integration test control register, TimerITCR

This is a single-bit register used to enable integration test mode. When in this mode, the masked interrupt outputs are directly controlled by the test output set register. The combined interrupt output TIMINTC then becomes the logical OR of the bits set in the test output set register. Table 4-33 shows the bit assignments for the TimerITCR register.

Table 4-33 TimerITCR register

Bits	Name	Type	Function
0	Integration Test Mode Enable	Read/write	When set HIGH, places the Timers into integration test mode

Integration test output set register, TimerITOP

When in integration test mode, the enabled interrupt outputs are driven directly from the values in this register. Table 4-34 shows the bit assignments for the TimerITOP register.

Table 4-34 TimerITOP register

Bits	Name	Type	Function
1	Integration Test TIMINT2 value	Write	Value output on TIMINT2 when in Integration Test Mode
0	Integration Test TIMINT1 value	Write	Value output on TIMINT1 when in Integration Test Mode

Peripheral identification registers, TimerPeriphID0-3

The TimerPeriphID0-3 registers are four 8-bit registers, that span address locations 0xFE0-0xFEC. The registers can conceptually be treated as a single 32-bit register. The read-only registers provide the following options of the peripheral:

Part number [11:0] This is used to identify the peripheral. The three digit product code 0x804 is used for the timer.

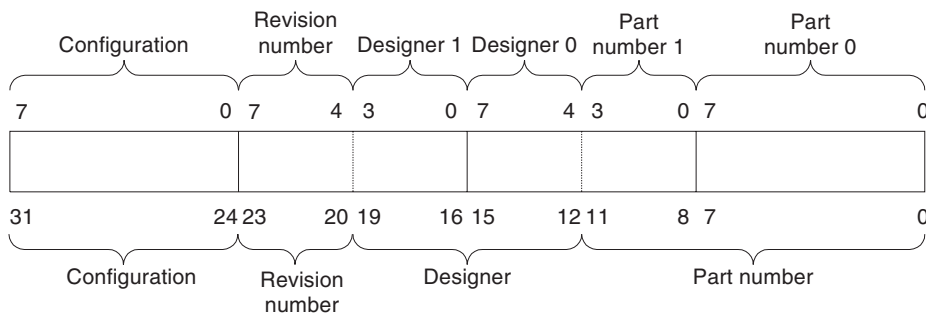
Designer [19:12] This is the identification of the designer. ARM Limited is 0x41 (ASCII A).

Revision number [23:20]
This is the revision number of the peripheral. The revision number starts from 0.

Configuration [31:24]
This is the configuration option of the peripheral. The configuration value is 0.

Figure 4-12 shows the bit assignment for the TimerPeriphID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 4-12 Peripheral identification register bit assignment

Note

When you design a systems memory map you must remember that the register has a 4KB-memory footprint.

The 4-bit revision number is implemented by instantiating a component called *RevisionAnd* four times with its inputs tied off as appropriate, and the output sent to the read multiplexor.

All memory accesses to the peripheral identification registers must be 32-bit, using the LDR and STR instructions.

The four, 8-bit peripheral identification registers are described in the following subsections:

- *TimerPeriphID0* register on page 4-34
- *TimerPeriphID1* register on page 4-34
- *TimerPeriphID2* register on page 4-34
- *TimerPeriphID3* register on page 4-35.

TimerPeriphID0 register

The TimerPeriphID0 register is hard coded and the fields within the register determine the reset value. Table 4-35 shows the bit assignment of the TimerPeriphID0 register.

Table 4-35 TimerPeriphID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:0	Partnumber0	Read	These bits read back as 0x04

TimerPeriphID1 register

The TimerPeriphID1 register is hard coded and the fields within the register determine the reset value. Table 4-36 shows the bit assignment of the TimerPeriphID1 register.

Table 4-36 TimerPeriphID1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:4	Designer0	Read	These bits read back as 0x1
3:0	Partnumber1	Read	These bits read back as 0x08

TimerPeriphID2 register

The TimerPeriphID2 register is hard coded and the fields within the register determine the reset value. Table 4-37 shows the bit assignment of the TimerPeriphID2 register.

Table 4-37 TimerPeriphID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:4	Revision	Read	These bits read back as 0x0
3:0	Designer1	Read	These bits read back as 0x4

TimerPeriphID3 register

The TimerPeriphID3 register is hard coded and the fields within the register determine the reset value. Table 4-38 shows the bit assignment of the TimerPeriphID3 register.

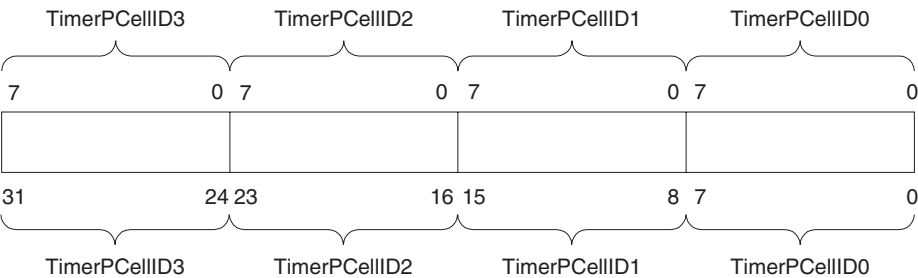
Table 4-38 TimerPeriphID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:0	Configuration	Read	These bits read back as 0x0

PrimeCell identification registers, TimerPCellID0-3

The TimerPCellID0-3 registers are four 8-bit registers, that span address locations 0xFF0-0xFFC. The read-only registers can conceptually be treated as a single 32-bit register. The register is used as a standard cross-peripheral identification system. Figure 4-13 shows the bit assignment for the TimerPCellID0-3 registers.

Actual register bit assignment



Conceptual register bit assignment

Figure 4-13 PrimeCell identification register bit assignment

The four, 8-bit registers are described in the following subsections:

- *TimerPCellID0 register* on page 4-36
- *TimerPCellID1 register* on page 4-36
- *TimerPCellID2 register* on page 4-36
- *TimerPCellID3 register* on page 4-36.

TimerPCellID0 register

The TimerPCellID0 register is hard coded and the fields within the register determine the reset value. Table 4-39 shows the bit assignment of the TimerPCellID0 register.

Table 4-39 TimerPCellID0 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:0	TimerPCellID0	Read	These bits read back as 0x0D

TimerPCellID1 register

The TimerPCellID1 register is hard coded and the fields within the register determine the reset value. Table 4-40 shows the bit assignment of the TimerPCellID1 register.

Table 4-40 TimerPCellID1 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:0	TimerPCellID1	Read	These bits read back as 0xF0

TimerPCellID2 register

The TimerPCellID2 register is hard coded and the fields within the register determine the reset value. Table 4-41 shows the bit assignment of the TimerPCellID2 register.

Table 4-41 TimerPCellID2 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:0	TimerPCellID2	Read	These bits read back as 0x05

TimerPCellID3 register

The TimerPCellID3 register is hard coded and the fields within the register determine the reset value. Table 4-42 shows the bit assignment of the TimerPCellID3 register.

Table 4-42 TimerPCellID3 register

Bits	Name	Type	Function
31:8	-	-	Reserved, read undefined, must read as zeros
7:0	TimerPCellID3	Read	These bits read back as 0xB1

4.5.5 Signal descriptions

Table 4-43 contains a list of non-AMBA signals used by the timer.

Table 4-43 Signal descriptions for timer

Signal	Type	Direction	Description
TIMCLK	Timer clock	Input	The timer clock input. This must be synchronous to PCLK for normal operation.
TIMCLKEN1	Timer 1 clock enable	Input	The enable for the timer1 clock input. The counter will only decrement on a rising edge of TIMCLK when TIMCLKEN1 is HIGH.
TIMCLKEN2	Timer 2 clock enable	Input	The enable for the timer clock input. The counter will only decrement on a rising edge of TIMCLK when TIMCLKEN2 is HIGH.
TIMINT1	Counter 1 interrupt	Output	Active HIGH interrupt signal to the interrupt controller module. This signal indicates an interrupt has been generated by counter 1 having being decremented to zero.
TIMINT2	Counter 2 interrupt	Output	Active HIGH interrupt signal to the interrupt controller module. This signal indicates an interrupt has been generated by counter 2 having being decremented to zero.
TIMINTC	Combined Counter Interrupt	Output	Active HIGH interrupt signal to the interrupt controller module. This signal indicates an interrupt has been generated by either counter having being decremented to zero, and is the logical OR of TIMINT1 and TIMINT2 .

———— **Note** ————

For a description of the AMBA signals used by the timer, see *AMBA signals* on page 1-4.

Chapter 5

Behavioral Models

This chapter describes the behavioral models in the *AMBA Design Kit* (ADK). It contains the following sections:

- *External RAM (ExtRAM)* on page 5-2
- *Internal memory (IntMem)* on page 5-3
- *External ROM (ExtROM)* on page 5-5
- *Tube (Tube)* on page 5-6
- *File reader bus master (FileReader)* on page 5-7
- *TIC driver (TIC)* on page 5-10.

5.1 External RAM (ExtRAM)

The external RAM module is a simple model of a 32K x 8 off-chip SRAM, that can be initialized with data from a local file. The external RAM module is shown in Figure 5-1.

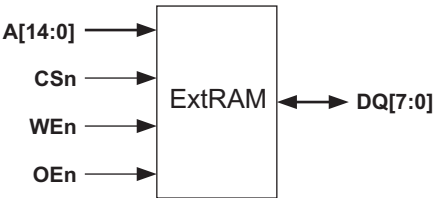


Figure 5-1 AHB external RAM module interface diagram

5.1.1 Programmer’s model

Table 5-1 shows the user-defined settings for the external RAM module.

Table 5-1 User-defined settings for the external RAM module

Signal	Type	Default setting	Description
RAMDEPTH	Memory depth	32	This sets the memory depth in KB. If the value is increased from the default setting, then the address input bus A must also be increased to allow all memory to be addressed.
FileName	Input filename	-	This points to the local input data file that is read in after reset.

5.1.2 Signal descriptions

Table 5-2 contains a list of non-AMBA signals used by the external RAM module.

Table 5-2 Signal descriptions for the external RAM module

Signal	Type	Direction	Description
A[14:0]	External address	Input	The external address input
DQ[7:0]	External data I/O	Input/output	The external data bus, sampled during write transfers and driven during read transfers
CSn	Chip enable	Input	When LOW this signal indicates that the chip has been selected and must respond to the current transfer
WEn	Write enable	Input	When LOW this signal indicates a write transfer
OEn	Output enable	Input	When LOW this signal indicates a read transfer, and enables the module to drive data onto DQ

5.2 Internal memory (*IntMem*)

The internal memory is a simple little-endian model of a variable-depth 32-bit wide on-chip SRAM, which can be initialized with data from a local file.

———— **Note** ————

Initialization is completed only once at simulation start up, and is not carried out based on the module reset signal.

The internal memory module is shown in Figure 5-2.

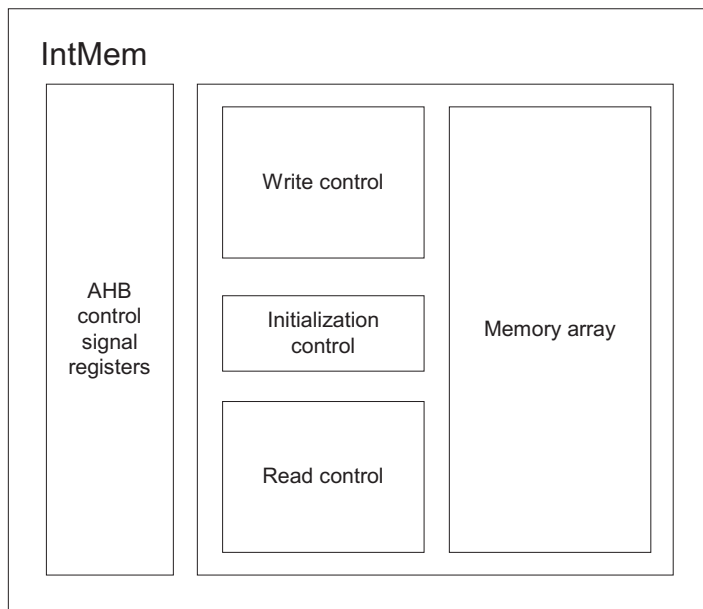


Figure 5-2 AHB internal memory module block diagram

5.2.1 Programmer’s model

Table 5-3 shows the user-defined settings for the internal memory module.

Table 5-3 User-defined settings for the internal RAM module

Signal	Type	Default setting	Description
MemBits	Memory address width	12	This sets the number of used address bits
FileName	Input filename	intram.dat	This points to the local input data file that is read in after reset

Memory initialization from local data file

On simulation initialization, the internal memory module loads in data from the file specified in the FileName setting. This must be stored as an 8-character Verilog \$readmemh format data file (for both VHDL and Verilog format models), which cannot contain more data than the model will support. Address lines (starting with @) and single line comments (starting with //) are valid, but all other non-value characters are not allowed. Loading starts from address zero, and continues incrementing on word boundaries until an address line is found in the file. Loading then continues from that address. All values are initialized to zero before loading is started. An example intram.dat file is shown in Example 5-1.

Example 5-1

```
ea00000b
ea000005
// Data values stored at 0x00000200
@00000200
01234567
89ABCDEF
```

5.2.2 Signal descriptions

The internal memory module uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

5.3 External ROM (*ExtROM*)

The external ROM module is a simple model of a 16K x 8 off-chip EPROM, which can be initialized with data from a local file. The external ROM module is shown in Figure 5-3.

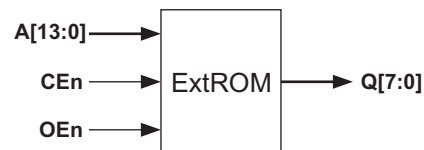


Figure 5-3 External ROM module interface diagram

5.3.1 Programmer's model

Table 5-4 shows the user-defined settings for the external ROM module

Table 5-4 User-defined settings for the external ROM module

Signal	Type	Default setting	Description
ROMDEPTH	Memory depth	16	This sets the memory depth in KB. If the value is increased from the default setting, then the address input bus A must also be increased to allow all memory to be addressed.
FileName	Input filename	-	This points to the local input data file that is read in after reset.

5.3.2 Signal descriptions

Table 5-5 contains a list of non-AMBA signals used by the external ROM module.

Table 5-5 Signal descriptions for the external ROM module

Signal	Type	Direction	Description
A[13:0]	External address	Input	The external address input
Q[7:0]	External data out	Output	The external data bus, driven during read transfers
CEn	Chip enable	Input	When LOW, this signal indicates that the chip has been selected and must respond to the current transfer
OEn	Output enable	Input	When LOW, this signal indicates a read transfer, and enables the module to drive data onto Q

5.4 Tube (Tube)

The tube is a simple method of passing system messages from a test program to the display, and allows a test program to stop the simulation.

Figure 5-4 shows the tube module interface.

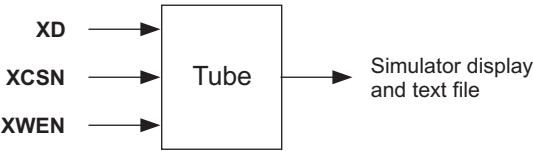


Figure 5-4 Tube module interface diagram

The main sections of this module are:

- message output to simulator
- message output to file
- simulation termination control.

5.4.1 Signal descriptions

Table 5-6 contains a list of non-AMBA signals used by the tube module.

Table 5-6 Signal descriptions for the tube module

Signal	Type	Direction	Description
XD[31:0]	External data	Input	This is the external data bus, which is sampled by this module during write transfers.
XCSN[3:0]	External chip select	Input	These signals are active LOW chip enables.
XWEN[3:0]	External write enable	Input	This is the active LOW memory write enable. For little-endian systems, XWEN[0] controls writes to the least significant byte and XWEN[3] , the most significant. The example system is configured to be little-endian.

5.5 File reader bus master (*FileReader*)

The AHB file reader bus master enables designers to simulate AMBA systems quickly and efficiently by using it to generate explicit bus transfers. The master can operate in the existing system with or without an ARM core present.

The file reader can be used as a *Bus Functional Model* (BFM) to exercise slave peripherals or generate bus traffic to help with AMBA infrastructure design. This facilitates arbiter algorithm design by being able to easily create multiple master systems where the designer has direct control over the bus activity.

The file reader is controlled through a simple scripting language which describes the transfers at a signal level. These are described in ASCII text and must be preprocessed for execution at simulation time.

Figure 5-5 shows a block diagram of the file reader bus master.

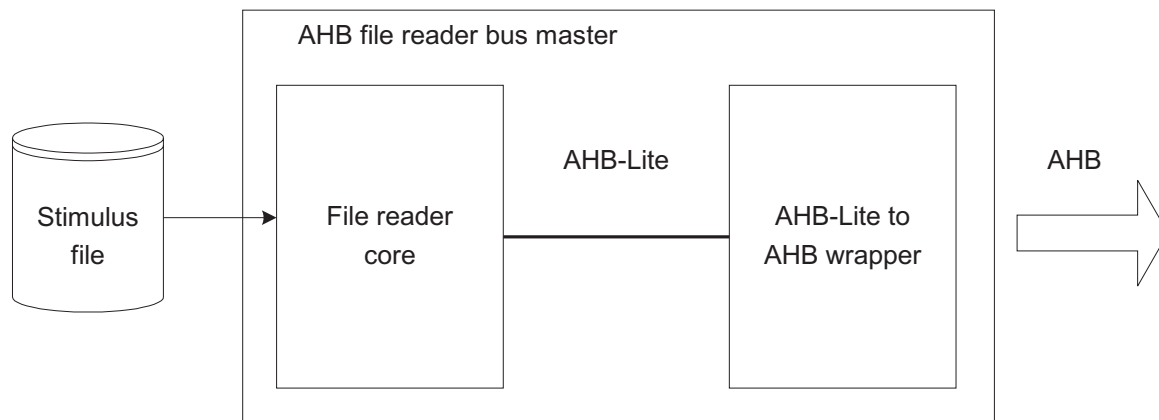


Figure 5-5 File reader bus master

5.5.1 Signal descriptions

The file reader bus master can be used in two ways

- as a full AHB bus master
- with the AHB-Lite interface (see Appendix A *AHB-Lite Overview* for more information).

Depending on which option you use, there are two signal options for the file reader bus master:

- *Top-level file reader bus master signals* on page 5-8
- *AHB-Lite file reader core signals* on page 5-8.

Top-level file reader bus master signals

The file reader bus master uses only AMBA signals. For a description of the AMBA signals, see *AMBA signals* on page 1-4.

AHB-Lite file reader core signals

Table 5-7 contains a list of the signals used by the AHB-Lite file reader core.

Table 5-7 Signal descriptions for AHB-Lite file reader core

Signal	Type	Direction	Description
MRDATA[31:0]	Read data bus	Input	AHB read data passed through the wrapper to the AHB-Lite interface. Current data corresponds to the previous address on MADDR .
MREADY	Transfer done	Input	When asserted by the AHB wrapper, indicates the current transfer has completed. When deasserted, the wrapper forces the transfer to be extended.
MERROR	Error response	Input	When asserted, indicates an error response from the addressed AHB slave.
MMASTLOCK	Locked sequence	Output	When HIGH indicates that a locked transfer is required. This signal must be asserted a cycle before MTRANS changes from IDLE to NONSEQUENTIAL.
MADDR[31:0]	Address bus	Output	Indicates the required address for the transfer. MADDR is subject to the same restrictions as HADDR (that is, address alignment).
MTRANS[1:0]	Transfer type	Output	Indicates the state of the current transfer. MTRANS assumes the same states as AHB signal HTRANS .

Table 5-7 Signal descriptions for AHB-Lite file reader core (continued)

Signal	Type	Direction	Description
MWRITE	Transfer direction	Output	Indicates a write operation when high and a read operation when LOW. MWRITE has the same function as HWRITE .
MSIZE[2:0]	Transfer size	Output	Indicates the active data lanes for the current transfer. MSIZE assumes the same states as AHB signal HSIZE .
MBURST[2:0]	Burst type	Output	Indicates the type of burst for the current transfer. MBURST assumes the states as AHB signal HBURST .
MPROT[3:0]	Protection control	Output	Protection control signals for the current transfer. MPROT assumes the states as AHB signal HPROT .
MWDATA[31:0]	Write data bus	Output	Data from the file reader, the current data corresponds to the previous address on MADDR .

Note

For a description of the AMBA signals used by the AHB-Lite file reader core, see *AMBA signals* on page 1-4.

5.6 TIC driver (TIC)

The test interface driver (Ticbox) is an external module which drives the test interface lines to gain access to the AHB bus, and then applies test vectors from a test input file. This test input file is the output from a C program written with the TICTalk command language.

Before reading this section, you should be familiar with AMBA and its test interface protocol. If not, refer to the *AMBA Specification* for further information. Figure 5-6 shows an interface diagram of the Ticbox module.

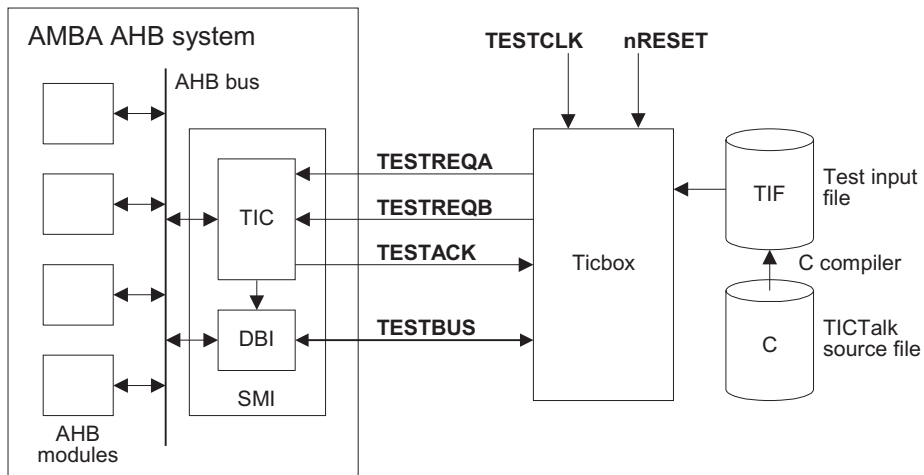


Figure 5-6 Ticbox module interface diagram

The main sections of this module are:

- the input file reader
- output vector generation
- read data expected value checking.

When the external system reset input has been deasserted, the Ticbox requests access to the system. This is done by asserting **TESTREQA** HIGH and **TESTREQB** LOW. The TIC then indicates when test mode has been entered by asserting **TESTACK** HIGH. When in test mode, the test input file is then read and translated by the Ticbox into AMBA test interface transactions, using the **TESTREQA** and **TESTREQB** signals.

The Ticbox applies test vectors to the system every time the **TESTACK** line indicates the system is ready. On read cycles the value is masked and then compared with the masked expected value given in the test vector file. An error message is given if the comparison fails. System testing ends once the end of the input vector file is reached, and the Ticbox indicates this by asserting both **TESTREQA** and **TESTREQB** LOW to end the simulation.

5.6.1 Signal descriptions

Table 5-8 contains a list of non-AMBA signals used by the Ticbox module.

Table 5-8 Signal descriptions for Ticbox module

Signal	Type	Direction	Description
TESTCLK	Test mode clock	Input	This is the system clock HCLK in test mode. All the test interface transactions are timed using this signal.
nReset	External reset	Input	Active LOW external reset input. Used to control the operation of the Ticbox module.
TESTREQA	Test request A	Output	Indicates test vector mode. Refer to the test interface chapter in the <i>AMBA Specification</i> for further information about the test protocol. It is driven early in the LOW phase of TESTCLK and held to the falling edge of TESTCLK .
TESTREQB	Test request B	Output	Indicates test vector mode. Refer to the test interface chapter of the <i>AMBA Specification</i> for further information about the test protocol. It is driven early in the LOW phase of TESTCLK and held to the falling edge of TESTCLK .
TESTACK	Test acknowledge	Input	Indicates that the test bus has been granted and also that a test access has been completed.
TESTBUS[31:0]	Test data bus	Input/output	32-bit bidirectional test port.

5.6.2 User-defined settings

Table 5-9 shows the user-defined settings for the Ticbox module.

Table 5-9 User-defined settings for the Ticbox module

Name	Type	Default setting	Description
FileName	Input filename	infile.tif (VHDL) infile.sim (Verilog)	This points to the local input vector file that is read in a line at a time as each vector is performed.
HaltOnMismatch	Read error setting	FALSE	This is used to control the operation of the module when a read error is detected. When set FALSE, a warning message will be displayed showing the read error, and if set TRUE, the simulation will be halted when a read error is detected.
Verbosity	Comment display	TRUE	Controls the displaying of input vector file comments. When set TRUE, comments are displayed, and when set FALSE, comments are not displayed. This does not affect the displaying of other system messages.

Chapter 6

PrimeCell GPIO

This chapter describes how the PrimeCell *General Purpose Input/Output* (GPIO) is integrated within the ADK. It contains the following sections:

- *Operation* on page 6-2
- *Integration within ADK* on page 6-3.

6.1 Operation

The PrimeCell GPIO is an AMBA compliant *System-on-Chip* (SoC) peripheral. It is an AMBA slave module that connects to the *Advanced Peripheral Bus* (APB) compliant with *AMBA Specification (Rev 2.0)* onwards.

The GPIO offers:

- compliance to the *AMBA Specification (Rev 2.0)* onwards for easy integration into *System-on-a-Chip* (SoC) implementation
- eight individually programmable input/output pins
- scalability by multiple instantiations to 16, 24, 32, 40, or more bits
- programmable interrupt generation capability, from a transition or a level condition, on any number of pins
- hardware control capability of PrimeCell GPIO lines for different system configurations
- bit masking in both read and write operations through address lines
- identification registers that uniquely identify the PrimeCell GPIO.

The direction registers for port A and port B are programmable. Additional test registers and modes are implemented for functional verification and manufacturing test.

All block registers are cleared during power-on reset (**PRESETn** LOW). This enables the input drivers for both ports A and B which default to inputs on reset.

For each port, there is a data register and a data direction register. On reads, the data register contains the current status of corresponding port pins, and whether they are configured as input or output. Writing to a data register only affects the pins that are configured as outputs.

Additional test registers and modes are implemented for functional verification and manufacturing test.

For full details of the features provided by the PrimeCell GPIO, see the *General Purpose Input/Output (PL061) Technical Reference Manual*.

6.2 Integration within ADK

For details on how the GPIO integrates with the ADK, see the *AMBA Design Kit User Guide*.

Appendix A

AHB-Lite Overview

This appendix provides an overview of the AHB-Lite. It contains the following sections:

- *Introduction* on page A-2
- *AHB-Lite master* on page A-4
- *AHB-Lite slaves* on page A-5.

A.1 Introduction

AHB-Lite is a subset of the full AHB specification and is intended for use in designs where only a single bus master is used. This can either be a simple single-master system, as shown below, or a Multi-layer AHB system where there is only one AHB master per layer.

Figure A-1 shows a block diagram of the single-master system.

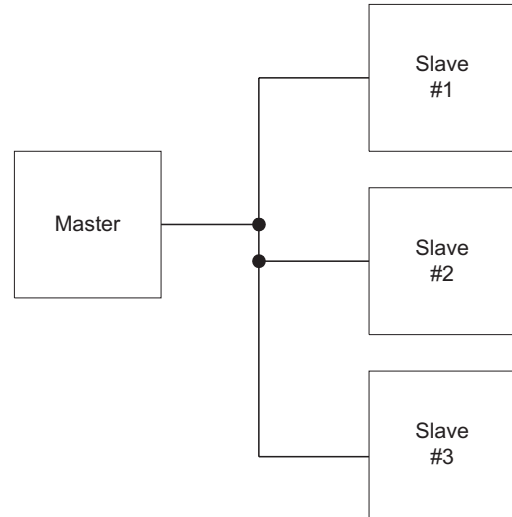


Figure A-1 AHB-Lite single-master system

AHB-Lite simplifies the AHB specification by removing the protocol required for multiple bus masters, which includes the Request/Grant protocol to the Arbiter and the Split/Retry responses from slaves.

Masters designed to the AHB-Lite interface specification can be significantly simpler in terms of interface design, compared to a full AHB master. AHB-Lite enables faster design and verification of these masters and the addition of a standard off-the-shelf bus mastering wrapper can be used to convert an AHB-Lite master for use in a full AHB system.

Any master that is already designed to the full AHB specification can be used in an AHB-Lite system with no modification.

The majority of AHB slaves can be used interchangeably in either an AHB or AHB-Lite system. This is because AHB slaves that do not use either the Split or Retry response are automatically compatible with both the full AHB and the AHB-Lite specification. It is only existing AHB slaves that do use Split/Retry responses that require an additional standard off-the-shelf wrapper to be used in an AHB-Lite system.

Any slave designed for use in an AHB-Lite system works in both a full AHB and an AHB-Lite design.

A.1.1 Specification

The AHB-Lite specification differs from the full AHB specification in the following ways:

- Only one master. There is only one source of address, control, and write data, so no Master-to-Slave multiplexor is required.
- No arbiter. None of the signals associated with the arbiter are used.
- Master has no **HBUSREQ** output. If such an output exists on a master, it is left unconnected.
- Master has no **HGRANT** input. If such an input exists on a master, it is tied HIGH.
- Slaves must not produce either a Split or Retry response.
- The AHB-Lite lock signal is the same as **HMASTLOCK** and it has the same timing as the address bus and other control signals. If a master has an **HLOCK** output, it can be retimed to generate **HMASTLOCK**.
- The AHB-Lite lock signal must remain stable throughout a burst of transfers, in the same way that other control signals must remain constant throughout a burst.

A.1.2 Compatibility

Table A-1 shows how masters and slaves designed for use in either full AHB or AHB-Lite can be used interchangeably in different systems.

Table A-1 AHB-Lite interchangeability

Component	Full AHB system	AHB-Lite system
Full AHB master	✓	✓
AHB-Lite master	Use standard AHB master wrapper	✓
AHB slave (no Split/Retry)	✓	✓
AHB slave with Split/Retry	✓	Use standard AHB slave wrapper

A.2 AHB-Lite master

An AHB-Lite master has the same signal interface as a full AHB bus master, except that it does not support **HBUSREQx** and **HGRANTx**.

The Lock functionality is still required because the master might be performing a transfer to a multi-port slave. The slave must be given an indication that no other transfer should occur to the slave when the master requires locked access.

An AHB-Lite master is not required to support either the Split or Retry response and only the OKAY and ERROR responses are required, so the AHB-Lite master interface does not require the **HRESP[1]** input.

A.2.1 AHB-Lite advantages

The advantage of using the AHB-Lite protocol is that the bus master does not have to support the following cases:

- Losing ownership of the bus. The clock enable for the master can simply be derived from the **HREADY** signal on the bus.
- Early terminated bursts. There is no requirement for the master to rebuild a burst due to early termination, because the master always has access to the bus.
- Split or Retry transfer responses. There is no requirement for the master to retain the address of the last transfer to be able to restart a previous transfer.

A.2.2 AHB-Lite conversion to full AHB

A standard wrapper is available to convert an AHB-Lite master to make it a full AHB master. This wrapper adds support for the features described above.

Because the AHB-Lite master has no bus request signal available, the wrapper generates this directly from the **HTRANS** signals.

A.3 AHB-Lite slaves

AHB slaves that do not use either the Split or Retry response can be used in either a full AHB or AHB-Lite system.

Any slave that does use Split or Retry responses can be used in an AHB-Lite system by simply adding a standard wrapper. This wrapper provides the ability to store the previous transfer in the case of a Split/Retry response and restart the transfer when appropriate. In fact, this wrapper is very similar to that required to convert an AHB-Lite master for use in a full AHB system.

For compatibility with Multi-layer AHB, it is required that all AHB-Lite slaves still retain support for early terminated bursts.

Figure A-2 on page A-6 shows a more detailed block diagram, including Decoder and Slave-to-Master multiplexor connections.

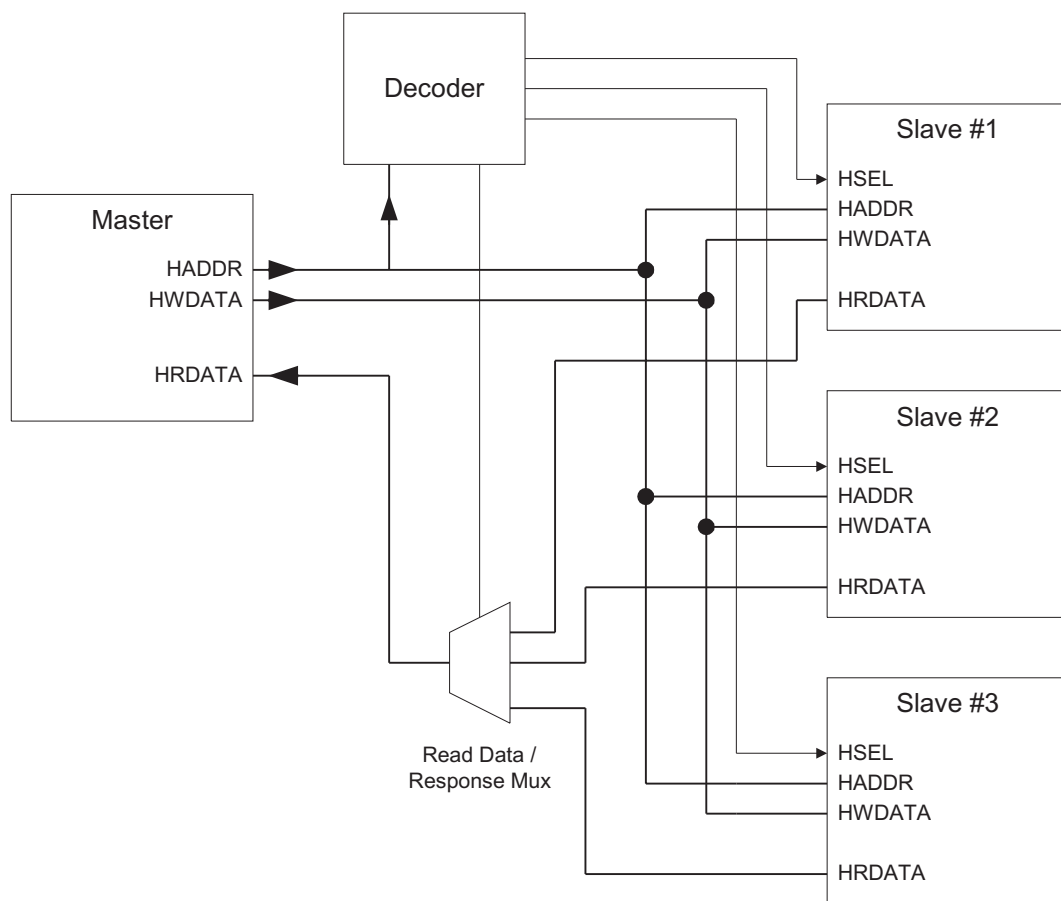


Figure A-2 AHB-Lite block diagram